



Docker

Tamara Crétard - 06.03.2026

Sommaire:

1. Contexte	3
2. Environnement de travail	3
2.1. Configuration utilisée	3
2.2. Schéma du réseau virtuel	4
3. Technologies et systèmes utilisés	5
3.1. Docker	5
3.2. Docker Hub	5
3.3. Docker Compose	5
3.4. Portainer	6
3.5. Jellyfin	6
3.6. OpenLDAP	6
4. Mise en oeuvre	7
4.1. Prérequis	7
4.2. Installation de Docker	8
4.3. Manipulation des images et conteneurs	10
4.3.1. Télécharger une image	10
4.3.2. Créer et gérer un conteneur	10
4.3.3. Interagir avec un conteneur	11
4.3.4. Supprimer un conteneur	12
4.4. Création d'une image personnalisée	12
4.4.1. Installer Apache2 dans un conteneur	13
4.4.2. Sauvegarder l'image avec docker commit	14
4.5. Installation et prise en main de Portainer	15
4.5.1. Installation de Portainer	15
4.5.2. Navigation dans l'interface	17
4.6. Création de stacks avec Docker Compose	18
4.6.1. Stack Nextcloud	18
4.6.2. Stack Nginx avec MariaDB et Adminer	24
4.7. Communication entre deux stacks	29
4.7.1. Mise en place de Jellyfin	30
4.7.1.1. Agrandissement du disque	30
4.7.1.2. Préparation des répertoires	34
4.7.1.3. Déploiement du stack Jellyfin	35
4.7.2. Déploiement du stack LDAP	45

4.7.2.1.	Création du réseau commun	45
4.7.2.2.	Déploiement du stack LDAP	46
4.7.2.3.	Création des utilisateurs dans phpLDAPadmin	49
4.7.3.	Connexion LDAP - Jellyfin	55
4.7.3.1.	Mise à jour du stack Jellyfin	55
4.7.3.2.	Installation et configuration du plugin LDAP dans Jellyfin.....	58
4.7.3.3.	Test de connexion avec un utilisateur LDAP	61
5.	Difficultés rencontrées.....	62
6.	Conclusion	63

1. Contexte

Dans le cadre d'un cours en BTS SIO, l'objectif de ce projet est de comprendre et maîtriser le fonctionnement de la conteneurisation d'applications. Pour cela, nous avons utilisé *Docker*, une solution open-source permettant d'emballer des applications dans des conteneurs légers et portables, ainsi que *Portainer* pour la gestion graphique de ces conteneurs. Également, la communication entre un stack *Jellyfin* (serveur multimédia) et un stack *OpenLDAP* (gestion centralisée des comptes utilisateurs) a été permise, permettant aux utilisateurs créés dans *OpenLDAP* de s'authentifier directement sur *Jellyfin*.

Ce travail a permis d'expérimenter l'installation complète de *Docker*, la manipulation d'images et de conteneurs, la création d'images personnalisées, le déploiement de stacks avec *Docker Compose*, ainsi que la mise en réseau de conteneurs. L'objectif final est de reproduire un fonctionnement similaire à celui d'une infrastructure d'entreprise moderne, où les applications sont déployées sous forme de conteneurs isolés et orchestrés.

2. Environnement de travail

Avant de commencer les différentes expérimentations avec *Docker*, il est essentiel d'analyser et de comprendre l'environnement de travail dans lequel le serveur et les machines clientes vont fonctionner.

2.1. Configuration utilisée

Tout d'abord, il est intéressant de connaître la configuration utilisée pour permettre les différentes expérimentations. Dans notre cas, la configuration utilisée est la suivante:

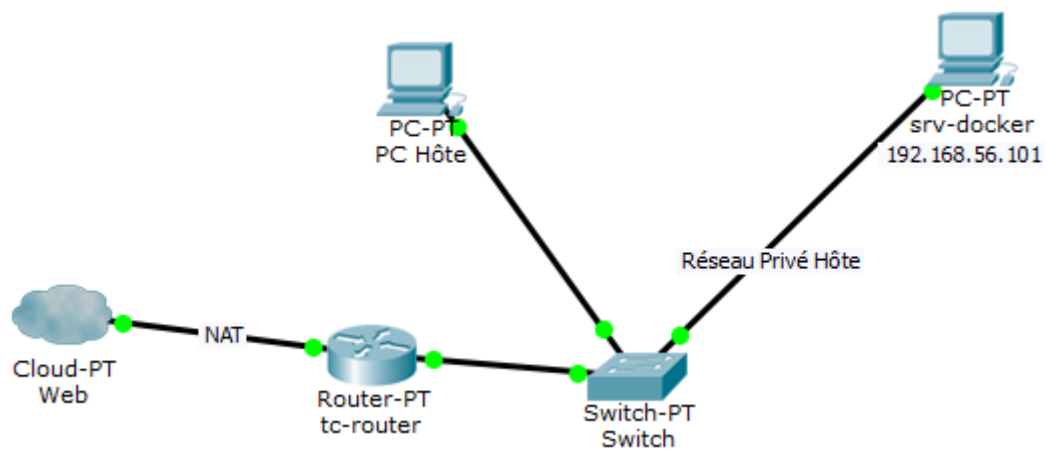
Processeur	11th Gen Intel(R) Core(TM) i7-11700K @ 3.60GHz 3.60 GHz
RAM	32 Go
Carte graphique	NVIDIA GeForce RTX 3080
SSD	2 To
Système d'exploitation	Microsoft Windows 11 Famille
Architecture	x64

2.2. Schéma du réseau virtuel

Afin de mieux visualiser notre installation, un schéma du réseau virtuel réalisé à l'aide de CISCO Packet Tracer est utile.

Nous avons besoin:

- d'une machine hôte
- d'une machine virtuelle Linux Debian *srv-docker* en Réseau Privé Hôte, en 192.168.56.101, sur laquelle sera installé *Docker* avec les différents stacks et containers.
- d'un routeur

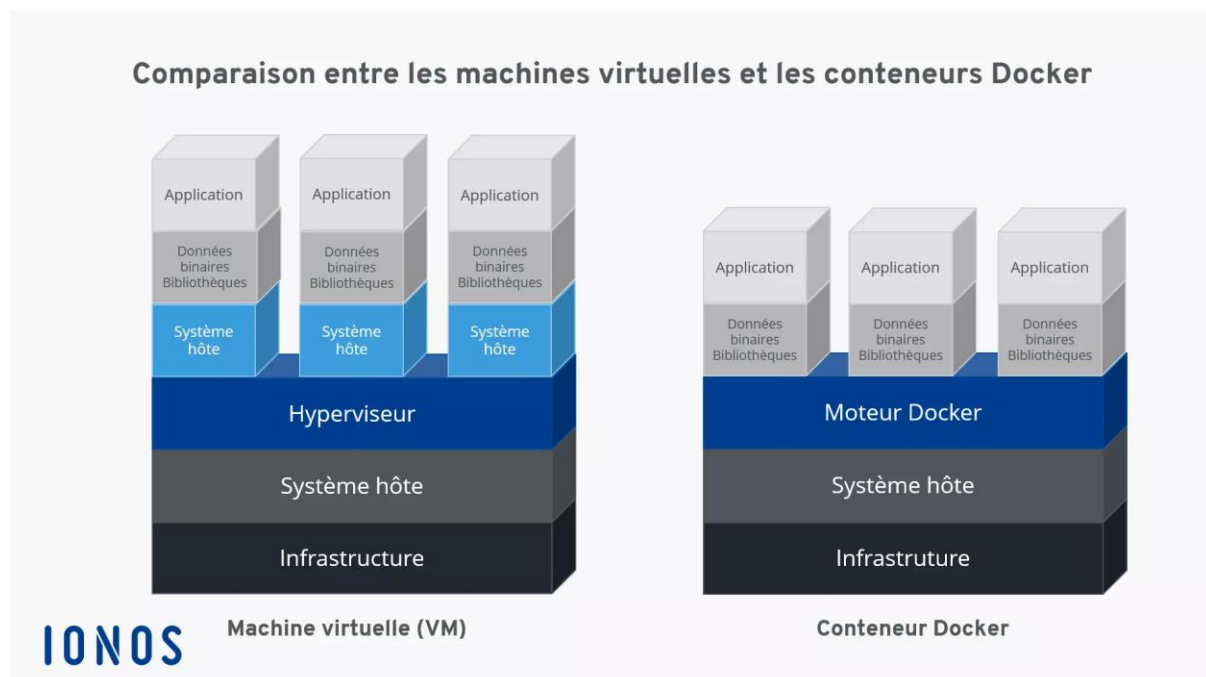


3. Technologies et systèmes utilisés

Différentes technologies et systèmes sont indispensables pour nos expérimentations. Il est donc nécessaire d'en comprendre le fonctionnement avant de les mettre en pratique.

3.1. Docker

Docker est une technologie open-source créée en 2013 par Solomon Hykes qui permet d'emballer une application et tout ce dont elle a besoin (bibliothèques, configuration, dépendances) dans un objet portable appelé conteneur. Contrairement à une machine virtuelle, un conteneur ne simule pas un système d'exploitation complet: il partage le noyau de la machine hôte, ce qui le rend beaucoup plus léger et rapide. Cela est très pratique pour déployer des applications de manière identique sur n'importe quelle machine, que ce soit en développement ou en production.



3.2. Docker Hub

Docker Hub est le registre officiel en ligne de *Docker*, créé par la société *Docker Inc.*, qui permet de stocker et partager des images de conteneurs. Une image est en quelque sorte de modèle figé à partir duquel on crée des conteneurs. Sur *Docker Hub*, on trouve des milliers d'images prêtes à l'emploi publiées par des entreprises ou des particuliers, comme Ubuntu, Nginx ou MySQL. *Docker Hub* peut être comparable à un App Store dédié aux conteneurs depuis lequel on peut télécharger une application en une seule commande.

3.3. Docker Compose

Docker Compose est un outil officiel de *Docker*, introduit en 2014 sous le nom de Fig avant d'être intégré au projet *Docker*, qui permet de définir et de gérer des applications

composées de plusieurs conteneurs. Plutôt que de lancer chaque conteneur manuellement avec de longues commandes, *Docker Compose* utilise un simple fichier de configuration appelé *docker-compose.yml* pour décrire l'ensemble des services, réseaux et volumes nécessaires à une application. En quelques secondes, tous les conteneurs de l'application démarrent automatiquement, dans le bon ordre et avec les bonnes dépendances entre eux. Cela est particulièrement utile pour des architectures modernes où une application web, sa base de données et son cache, par exemple, doivent fonctionner ensemble de manière cohérente et reproductible sur n'importe quel environnement.

3.4. Portainer

Portainer est une interface web open-source créée en 2016 qui permet de gérer ses conteneurs Docker sans passer par la ligne de commande. Depuis un navigateur, l'utilisateur peut démarrer, arrêter, supprimer ou surveiller ses conteneurs facilement. *Portainer* s'installe lui-même sous forme de conteneur *Docker*, ce qui le rend très simple à déployer. Il est particulièrement utile pour les débutants ou pour administrer un serveur à distance de façon visuelle et intuitive.

3.5. Jellyfin

Jellyfin est un serveur multimédia open-source créé en 2018 qui permet de centraliser films, séries, musiques et photos sur une seule machine. Il les rend ensuite accessibles depuis n'importe quel appareil du réseau local, comme un téléviseur, un téléphone ou un navigateur web. Contrairement à des services comme Netflix ou Plex, *Jellyfin* est entièrement gratuit, sans abonnement ni envoi de données vers un cloud tiers. Il se déploie très facilement via *Docker*, c'est pour cela qu'il est l'une des applications les plus populaires chez les amateurs de homelab.

3.6. OpenLDAP

LDAP (Lightweight Directory Access Protocol) est un protocole standard ouvert utilisé pour accéder à et gérer des annuaires d'informations distribués. Concrètement, il permet de centraliser la gestion des comptes utilisateurs: plutôt que de créer un compte par application, on crée un seul compte dans l'annuaire LDAP, qui peut ensuite être utilisé pour s'authentifier sur plusieurs services.

Dans le cadre de ce projet, nous utilisons *OpenLDAP*, une implémentation open-source du protocole LDAP, déployée via *Docker*. Pour faciliter la gestion des utilisateurs et des groupes, nous utilisons également *phpLDAPadmin*, une interface web qui permet d'administrer l'annuaire LDAP graphiquement, sans avoir à saisir de commandes complexes.

4. Mise en oeuvre

Maintenant que les différentes technologies et systèmes utilisés ont été présentés, nous pouvons passer à leur mise en œuvre concrète.

4.1. Prérequis

Dans un premier temps, il est nécessaire d'importer une machine Debian dans VirtualBox (en Réseau Privé Hôte), ainsi qu'un routeur (une interface en NAT et l'autre en Réseau Privé Hôte).

Une fois cela fait, les étapes suivantes permettent de préparer la machine virtuelle et l'environnement VirtualBox avant de pouvoir installer *Docker* et poursuivre nos expérimentations:

Etape	Description
1	Allumer la machine, puis entrer la commande nano /etc/hostname pour changer le nom de la machine.
	Représentation
	<code>root@srv-home:~# nano /etc/hostname</code>
Etape	Description
2	Entrer le nom de la machine pour qu'elle soit reconnaissable et ainsi ne pas se tromper dans les manipulations. Sauvegarder avec Ctrl+O puis Ctrl+X.
	Représentation
	<code>srv-docker</code>
Etape	Description
3	Entrer la commande nano /etc/network/interfaces .
	Représentation
	<code>root@srv-home:~# nano /etc/network/interfaces</code>

Etape	Description
4	Entrer l'adresse IP statique 192.168.56.101 avec un masque /24, puis ajouter la passerelle 192.168.56.254 qui correspond au routeur. Sauvegarder avec Ctrl+O puis Ctrl+X.
	Représentation
	<pre># The primary network interface allow-hotplug enp0s3 iface enp0s3 inet static address 192.168.56.101/24 gateway 192.168.56.254</pre>
Etape	Description
5	Redémarrer la machine avec reboot .
	Représentation
	<pre>root@srv-home:~# reboot</pre>

4.2. Installation de Docker

Maintenant que la machine *srv-docker* est correctement configurée, l'installation de *Docker* est possible:

Etape	Description
1	Mettre à jour les paquets avec la commande apt update .
	Représentation
	<pre>root@srv-docker:~# apt update</pre>
Etape	Description
2	Entrer la commande wget https://get.docker.com/ pour récupérer le script d'installation de docker (wget) dans le site web spécifié.
	Représentation
	<pre>root@srv-docker:~# wget https://get.docker.com/</pre>

Etape	Description
3	Mettre à jour la liste des paquets avec sed -i 's/bullseye\updates/bullseye-security/g' /etc/apt/sources.list
	Représentation
	<pre>root@srv-docker:~# sed -i 's/bullseye\updates/bullseye-security/g' /etc/apt/sources.list</pre>
Etape	Description
4	Exécuter le script d'installation téléchargé avec la commande bash index.html
	Représentation
	<pre>root@srv-docker:~# bash index.html</pre>
Etape	Description
5	Vérifier que Docker est actif en exécutant la commande systemctl status docker .
	Représentation
	<pre>root@srv-docker:~# systemctl status docker ● docker.service - Docker Application Container Engine Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled) Active: active (running) since Tue 2026-04-21 16:13:49 CEST; 21s ago TriggeredBy: ● docker.socket Docs: https://docs.docker.com Main PID: 8362 (dockerd) Tasks: 8 Memory: 30.5M CPU: 409ms CGroup: /system.slice/docker.service └─8362 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock</pre>
Etape	Description
6	Exécuter l'image <i>hello-world</i> pour contrôler que tout va bien avec docker run hello-world .
	Représentation
	<pre>root@srv-docker:~# docker run hello-world Hello from Docker! This message shows that your installation appears to be working correctly.</pre>

Il est possible de vérifier la liste des images que l'on a au sein de notre machine avec la commande **docker image ls**:

```
root@srv-docker:~# docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	f9078146db2e	25.9kB	9.49kB	U

4.3. Manipulation des images et conteneurs

4.3.1. Télécharger une image

Pour installer une image (ici *dokuwiki*), il suffit d'utiliser la commande **docker pull**. Dans cette commande, **pull** permet de récupérer l'image, **mprasil** est le nom du créateur et **dokuwiki** est le nom de l'image:

```
root@srv-docker:~# docker pull mprasil/dokuwiki
```

Il est possible de vérifier que l'image est présente sur la machine de nouveau avec la commande **docker image ls**:

```
root@srv-docker:~# docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	f9078146db2e	25.9kB	9.49kB	U
mprasil/dokuwiki:latest	507db13c7bda	511MB	125MB	

4.3.2. Créer et gérer un conteneur

A présent, nous allons d'abord créer un conteneur à partir de l'image *dokuwiki*. Pour cela, il est nécessaire d'entrer la commande **docker run -d -p 8001:80 --name wiki01 mprasil/dokuwiki**:

```
root@srv-docker:~# docker run -d -p 8001:80 --name wiki01 mprasil/dokuwiki
d37220625ea2fe23daed08202df0d61759584c31566561d1f47bfddb66dddc96
```

Pour cette commande, **run** signifie que le conteneur va être créé et rendu actif.

Également, **-d** (daemon) indique que l'on va lancer le conteneur en arrière plan.

Ensuite, **-p 8001:80** redirige le port 8001 de la machine hôte vers le port 80 du conteneur. Comme on peut lancer autant de conteneurs que l'on veut avec cette image, on fait une redirection de port sur un port préféré. A distance, on accédera au site sur le port choisi.

Enfin, **--name** permet de donner un nom au conteneur. Il faut aussi mentionner le nom de l'image que l'on va utiliser.

Après cela, il est possible d'observer la liste des conteneurs avec **docker container ls -a**:

```
root@srv-docker:~# docker container ls -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
d37220625ea2   mprasil/dokuwiki  "/startup.sh run"       18 seconds ago Up 18 seconds  0.0.0.0:8001->80/tcp,
[::]:8001->80/tcp  wiki01
89e5af8b82fd   hello-world     "/hello"                7 minutes ago  Exited (0) 7 minutes ago
festive_jackson
```

Cette commande avec l'option **-a** permet d'afficher tous les conteneurs, y compris ceux qui sont arrêtés.

4.3.3. Interagir avec un conteneur

Maintenant que le conteneur est créé, il est possible d'interagir avec lui. Pour ce faire, il est nécessaire d'entrer la commande **docker exec -it wiki01 bash**:

```
root@srv-docker:~# docker exec -it wiki01 bash
```

Pour cette commande, **exec** permet d'exécuter des lignes de commande à l'intérieur même du conteneur. L'option **-it**, elle, permet d'ouvrir un terminal interactif et **wiki01** est le nom du conteneur avec lequel on souhaite interagir. Enfin, **bash** exécute le shell bash à l'intérieur du conteneur.

Une fois dans le shell, il est possible d'utiliser **ls** pour voir le contenu du conteneur:

```
root@d37220625ea2:/# ls
bin  dev  dokuwiki.tgz  home  lib32  libx32  mnt  proc  run  srv  sys  usr
boot  dokuwiki  etc  lib  lib64  media  opt  root  sbin  startup.sh  tmp  var
```

Il est important de noter qu'un conteneur peut être à l'arrêt, il faut alors le démarrer avec **docker start** suivi du nom du conteneur.

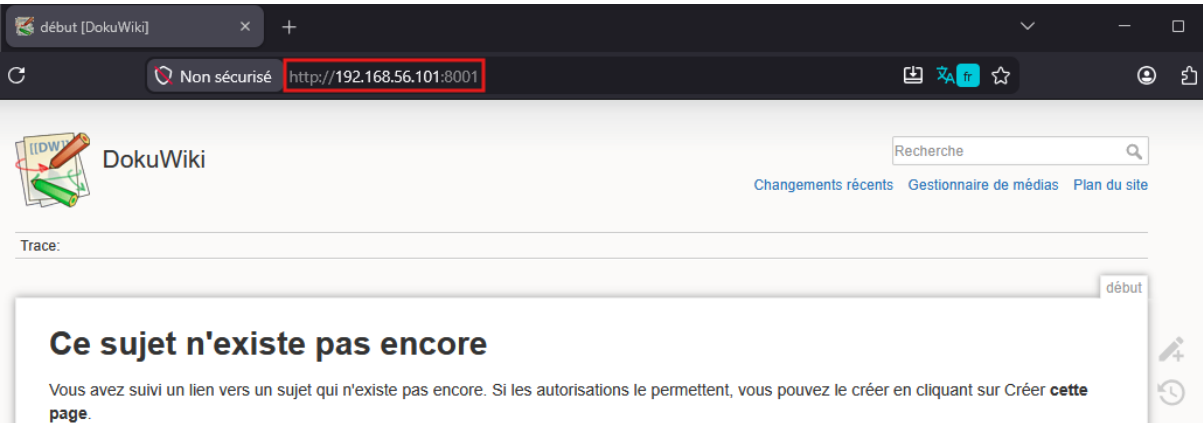
Également, pour sortir du mode d'exécution, il faut simplement entrer la commande **exit**:

```
root@d37220625ea2:/# exit
exit
```

Plusieurs conteneurs peuvent être créées à partir d'une seule image, il suffit simplement d'indiquer un port différent :

```
root@srv-docker:~# docker run -d -p 8002:80 --name wiki02 mprasil/dokuwiki
5da840e4e7e5d525a6ef55d60bd7f2ab80e1ec130d860e5579d9cdfddbb7a0d1
root@srv-docker:~# docker run -d -p 8003:80 --name wiki03 mprasil/dokuwiki
3ab2af7f3fd410bc7dde5b6a75dc7de8103b56cb04d80ce14e5f04b4de4d3767
```

Enfin, dans le navigateur, pour accéder au conteneur on entre l'adresse IP de la machine suivie du numéro de port:



4.3.4. Supprimer un conteneur

Après avoir créé un conteneur et interagit avec, s'il n'est plus utile, il est normal de vouloir le supprimer. Pour y arriver il faut:

Etape	Description
1	Stopper le conteneur, s'il est en cours de fonctionnement, avec la commande docker stop [nom du conteneur] ou [id du conteneur] .
	Représentation
	<pre>root@srv-docker:~# docker stop wiki01 wiki01</pre>
Etape	Description
2	Supprimer le conteneur avec la commande docker rm [nom du conteneur] .
	Représentation
	<pre>root@srv-docker:~# docker rm wiki01 wiki01</pre>

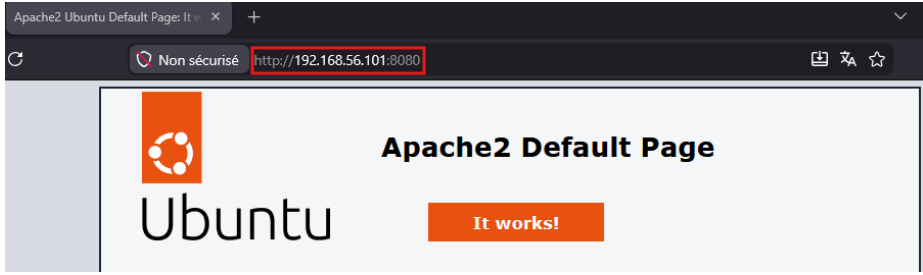
4.4. Création d'une image personnalisée

Il est possible de créer sa propre image *Docker* à partir d'un conteneur modifié. Dans cette section, nous allons installer Apache2 dans un conteneur Ubuntu, puis sauvegarder cet état sous forme d'une nouvelle image.

4.4.1. Installer Apache2 dans un conteneur

Afin d'installer Apache2 dans un conteneur, il y a plusieurs étapes à suivre:

Etape	Description
1	Mettre à jour les paquets avec la commande apt update .
	Représentation
	<pre>root@srv-docker:~# apt update</pre>
Etape	Description
2	Télécharger l'image officielle Ubuntu avec docker pull ubuntu .
	Représentation
	<pre>root@srv-docker:~# docker pull ubuntu</pre>
Etape	Description
3	Lancer un conteneur interactif depuis cette image avec docker run -ti -p 8080:80 ubuntu bash .
	Représentation
	<pre>root@srv-docker:~# docker run -ti -p 8080:80 ubuntu bash</pre>
Etape	Description
4	À l'intérieur du conteneur, installer Apache2 avec apt install apache2 -y
	Représentation
	<pre>root@f226cd4f5293:/# apt install apache2 -y</pre>

Etape	Description
5	Démarrer le service avec service apache2 start .
	Représentation
	<pre>root@f226cd4f5293:/# service apache2 start</pre>
Etape	Description
6	Vérifier que la page par défaut d'Apache2 est accessible depuis un navigateur à l'adresse <i>http://192.168.110.10:8080</i> .
	Représentation
	

4.4.2. Sauvegarder l'image avec docker commit

Une fois Apache2 installé et fonctionnel dans le conteneur, on sauvegarde l'état du conteneur sous forme d'une nouvelle image personnalisée :

```
root@srv-docker:~# docker commit beautiful_hoover my_ubuntu_apache
```

On peut ensuite créer un nouveau conteneur basé sur cette image personnalisée :

```
root@srv-docker:~# docker run -dit -p 8004:80 --name ubuntu_apache my_ubuntu_apache
d841f723f7bfe16bdfee3c6ac7d92a8807ea7e629f9564edc2a7706bb160d701
```

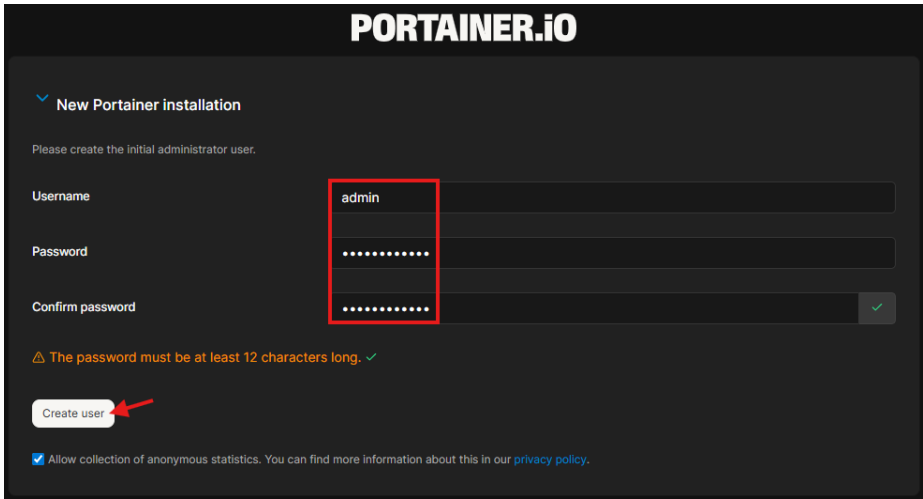
4.5. Installation et prise en main de Portainer

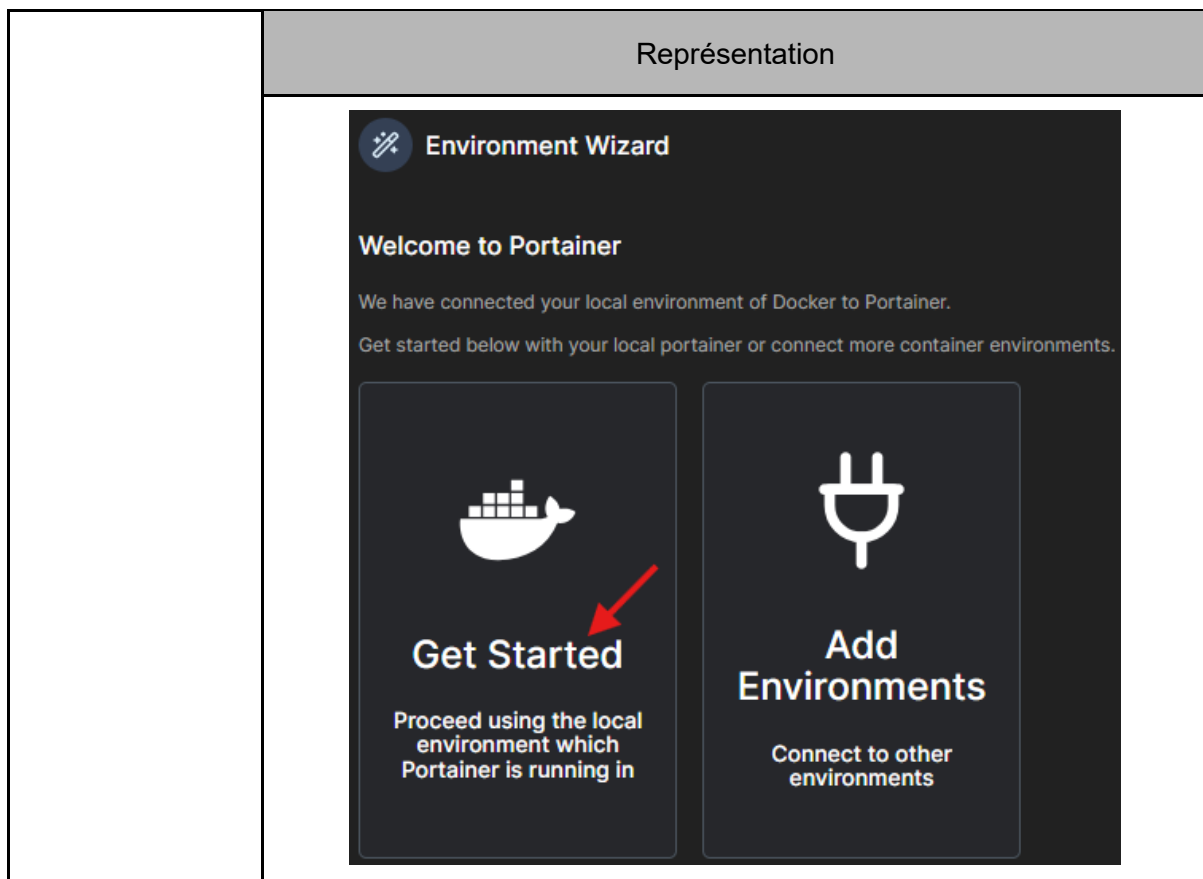
A présent, nous allons installer et utiliser *Portainer* qui est une interface web open source permettant de créer, modifier, redémarrer, surveiller des conteneurs Docker.

4.5.1. Installation de Portainer

Afin d'installer *Portainer*, il est nécessaire de passer par diverses étapes:

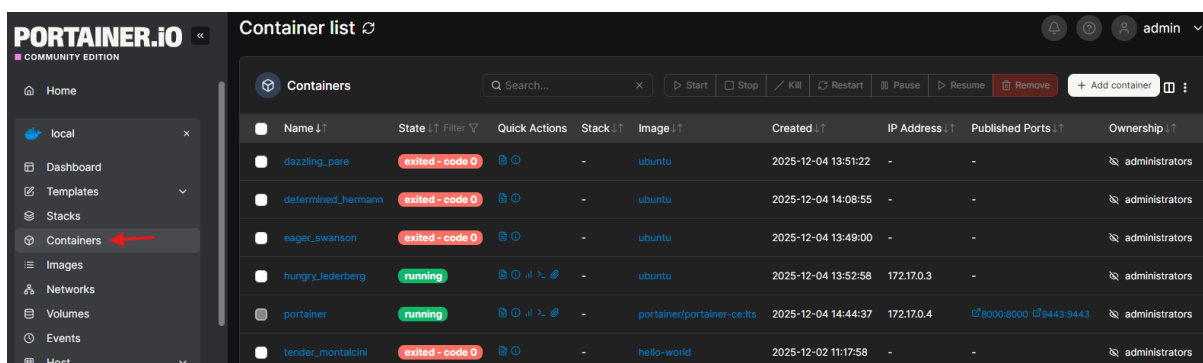
Etape	Description
1	Mettre à jour les paquets avec la commande apt update .
	Représentation
	<pre>root@srv-docker:~# apt update</pre>
Etape	Description
2	Installer <i>Docker Compose</i> pour créer des stacks avec la commande apt-get install docker-compose-plugin .
	Représentation
	<pre>root@srv-docker:~# apt-get install docker-compose-plugin</pre>
Etape	Description
3	Vérifier la version avec docker compose version .
	Représentation
	<pre>root@srv-docker:~# docker compose version Docker Compose version v5.1.3</pre>
Etape	Description
4	Créer un volume que <i>Portainer</i> utilisera pour stocker sa base de données avec docker volume create portainer_data
	Représentation
	<pre>root@srv-docker:~# docker volume create portainer_data portainer_data</pre>

Etape	Description
5	Télécharger et installer le conteneur <i>Portainer Server</i> . Il est nécessaire de faire un copier coller à partir du site https://docs.portainer.io/start/install-ce/server/docker/linux
	Représentation
	<pre>root@srv-docker:~# docker run -d -p 8000:8000 -p 9443:9443 --name portainer --restart=always -v /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce:sts</pre>
Etape	Description
6	Utiliser docker ps pour vérifier la présence du nouveau container.
	Représentation
	<pre>root@srv-docker:~# docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS 88ace66b4dd1 portainer/portainer-ce:sts "/portainer" 28 seconds ago Up 27 seconds 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9000/tcp portainer</pre>
Etape	Description
7	Accéder à <i>Portainer</i> via le navigateur avec l'adresse IP de la machine suivie du port du conteneur, dans notre cas 192.168.56.101:9443. Créer ensuite un utilisateur avec son mot de passe.
	Représentation
	
Etape	Description
8	Cliquer sur "Get Started".

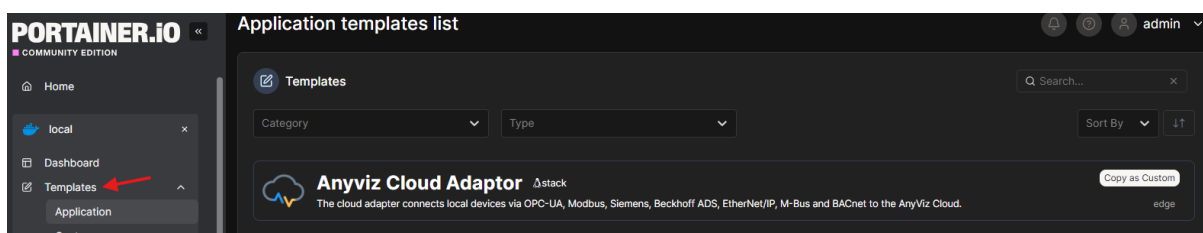


4.5.2. Navigation dans l'interface

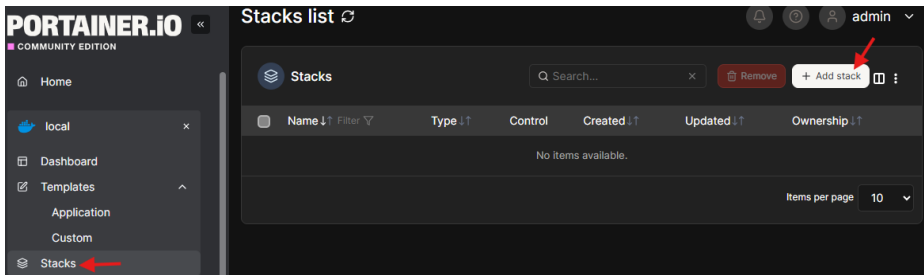
Une fois *Portainer* installé, la page d'accueil s'affiche. Il est alors possible d'observer la liste de tous les conteneurs installés grâce à l'onglet "Containers":



Un autre onglet important est l'onglet "Templates" qui permet de récupérer des images:



L’onglet “Stacks”, lui, permet de créer des stacks, c’est à dire simplement un ensemble de conteneurs qui fonctionnent ensemble pour faire tourner une application:



4.6. Création de stacks avec Docker Compose

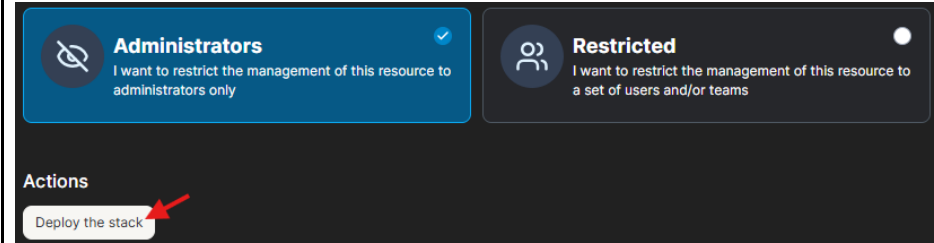
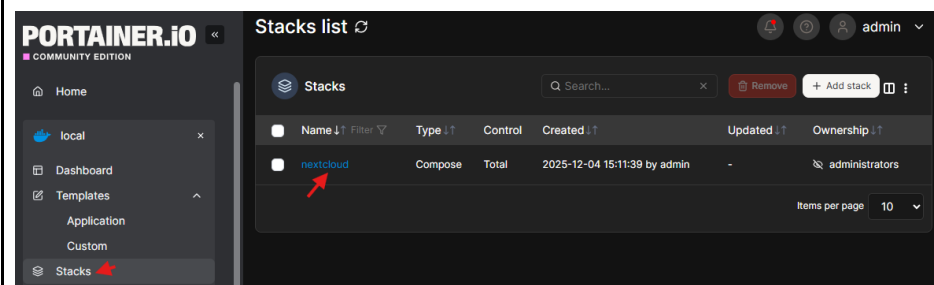
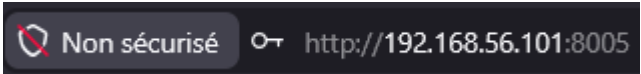
Docker Compose permet de définir et gérer des applications multi-conteneurs à l'aide d'un fichier YAML appelé stack. Dans *Portainer*, les stacks sont gérées directement depuis l'interface web.

4.6.1. Stack Nextcloud

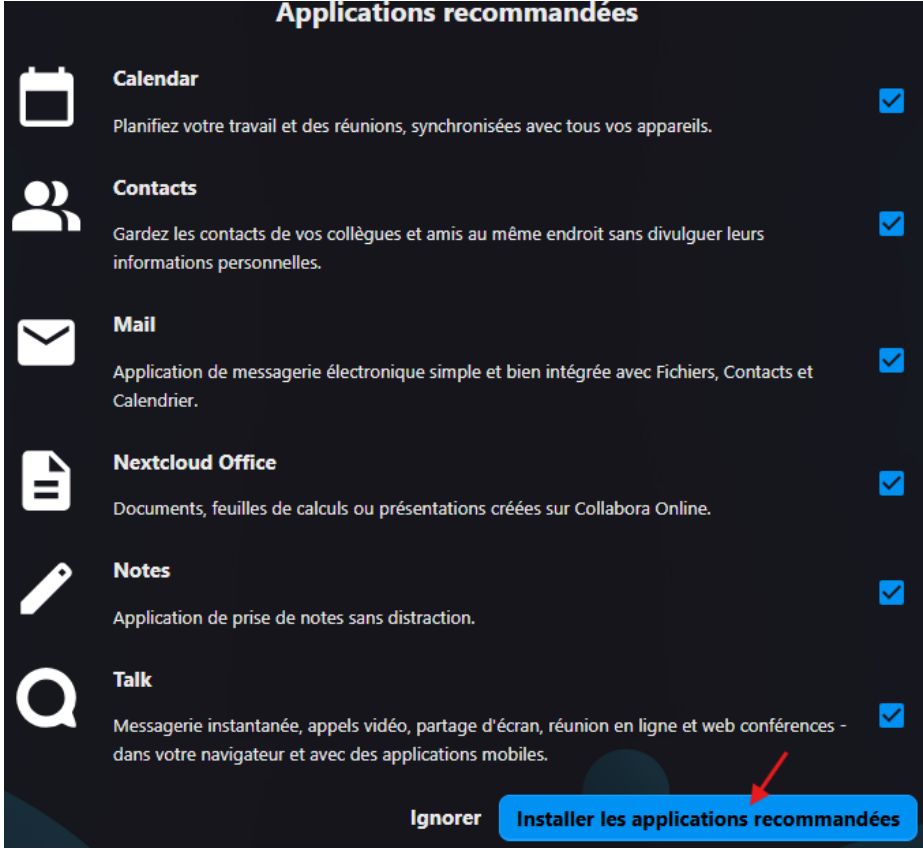
Nextcloud est un logiciel libre de stockage et partage de fichiers en ligne, offrant également des outils de travail collaboratif. Pour le déployer via *Portainer*, voici les étapes à suivre:

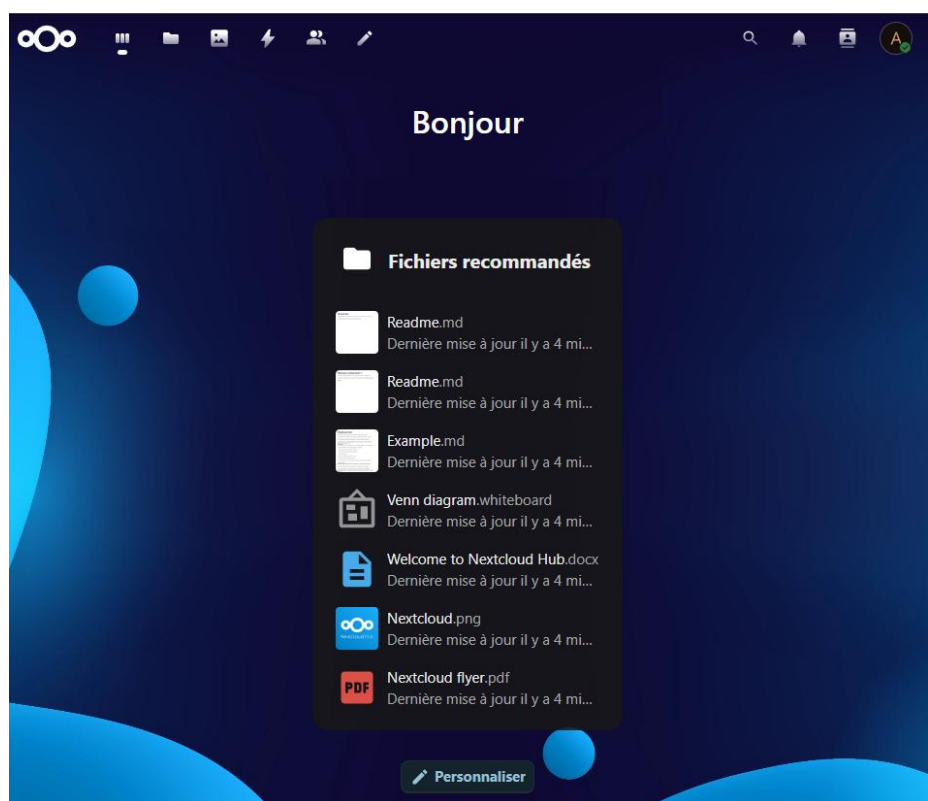
Etape	Description
1	Sur dockerhub, on trouve <i>NextCloud</i> : https://hub.docker.com/_/nextcloud . Copier le code qui s’y trouve.
	Représentation
	<pre>volumes: nextcloud: db: services: db: image: mariadb:10.6 restart: always command: --transaction-isolation=READ-COMMITTED --log-bin=binlog --binlog-format=R OW volumes: - db:/var/lib/mysql environment: - MYSQL_ROOT_PASSWORD= - MYSQL_PASSWORD= - MYSQL_DATABASE=nextcloud - MYSQL_USER=nextcloud app: image: nextcloud restart: always ports: - 8080:80 links: - db volumes: - nextcloud:/var/www/html environment: - MYSQL_PASSWORD= - MYSQL_DATABASE=nextcloud - MYSQL_USER=nextcloud - MYSQL_HOST=db</pre>

Etape	Description
2	Sur <i>Portainer</i> , aller dans l'onglet "Stacks", puis cliquer sur "Add stack".
	Représentation The screenshot shows the Portainer.io 'Stacks list' page. On the left sidebar, the 'Stacks' menu item is highlighted with a red arrow. In the main content area, the 'Add stack' button is highlighted with a red arrow.
Etape	Description
3	Donner un nom au stack et coller le code copié précédemment depuis <i>Docker Hub</i> .
	Représentation The screenshot shows the 'Add stack' form in Portainer.io. The 'Name' field is set to 'nextcloud'. The 'Build method' is set to 'Web editor'. The Docker Compose code is pasted into the editor: <pre> volumes: nextcloud: db: services: db: image: mariadb:10.6 restart: always command: --transaction-isolation=READ-COMMITTED --log-bin=binlog --binlog-format=ROW volumes: - db:/var/lib/mysql environment: - MYSQL_ROOT_PASSWORD= - MYSQL_PASSWORD= - MYSQL_DATABASE=nextcloud - MYSQL_USER=nextcloud </pre>

Etape	Description
4	Déployer le stack.
	Représentation
	
Etape	Description
5	Vérifier la création du stack dans l'onglet "Stacks".
	Représentation
	
Etape	Description
6	Dans le navigateur, se rendre à l'adresse du conteneur <i>Nextcloud</i> , choisir un nom d'administrateur et un mot de passe. Sélectionner <i>SQLite</i> , puis cliquer sur "Installer".
	Représentation
	

	Créer un compte administrateur Nom du compte administrateur admin Mot de passe du compte administrateur ⚠ Mot de passe moyen ▼ Stockage & base de données Répertoire des données /var/www/html/data Configuration de la base de données SQLite MySQL/MariaDB PostgreSQL ⚠ Avertissement à propos des performances Vous avez choisi SQLite comme base de données. SQLite ne devrait être utilisée que pour des instances minimales ou de développement. Pour une instance de production, nous recommandons une infrastructure de base de données différente. Si vous utilisez des clients de synchronisation de fichiers, l'utilisation de SQLite est fortement découragée. Installer → ℹ Besoin d'aide ? Lire la documentation
--	---

Etape	Description
7	Sur la nouvelle page, cliquer sur “Installer les applications recommandées”.
	Représentation
	 Applications recommandées Calendar Planifiez votre travail et des réunions, synchronisées avec tous vos appareils. ☒ Contacts Gardez les contacts de vos collègues et amis au même endroit sans divulguer leurs informations personnelles. ☒ Mail Application de messagerie électronique simple et bien intégrée avec Fichiers, Contacts et Calendrier. ☒ Nextcloud Office Documents, feuilles de calculs ou présentations créées sur Collabora Online. ☒ Notes Application de prise de notes sans distraction. ☒ Talk Messagerie instantanée, appels vidéo, partage d'écran, réunion en ligne et web conférences - dans votre navigateur et avec des applications mobiles. ☒ Ignorer Installer les applications recommandées

Etape	Description
8	Suite à cela, apparaît alors la page d'accueil de <i>Nextcloud</i> .
	Représentation
	

Si l'on souhaite supprimer le stack nextcloud, son image et les volumes, il faut d'abord taper la commande **docker volume ls | grep nextcloud**:

```
root@srv-docker:~# docker volume ls | grep nextcloud
local      nextcloud_db
local      nextcloud_nextcloud
```

docker volume ls permet de lister les volumes et **grep nextcloud** avec le pipeline permet de filtrer pour ne chercher que les résultats qui contiennent nextcloud.

Ensuite, il suffit de taper **docker volume rm [nom du volume]** pour le supprimer:

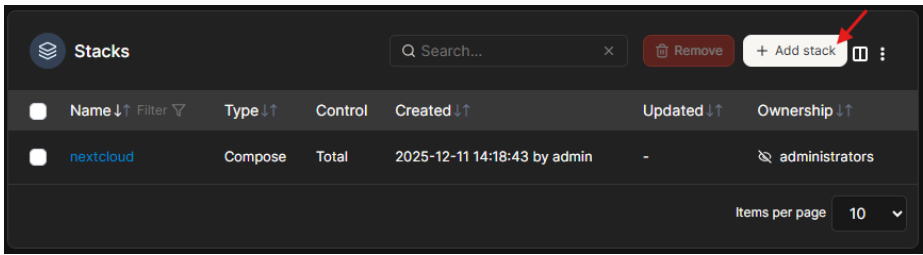
```
root@srv-docker:~# docker volume rm nextcloud_db
nextcloud_db
root@srv-docker:~# docker volume rm nextcloud_nextcloud
nextcloud_nextcloud
```


Enfin, on tape la commande **docker ps -a | grep nextcloud** pour vérifier que tout le contenu en lien avec nextcloud a été supprimé:

```
root@srv-docker:~# docker ps -a | grep nextcloud
```

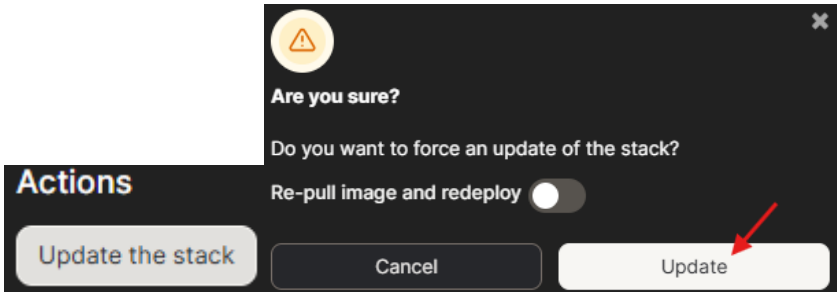
4.6.2. Stack Nginx avec MariaDB et Adminer

A présent, nous allons créer un stack qui combine un serveur web Nginx, une base de données MariaDB et l'interface d'administration Adminer. Pour cela, il est nécessaire de suivre les étapes suivantes:

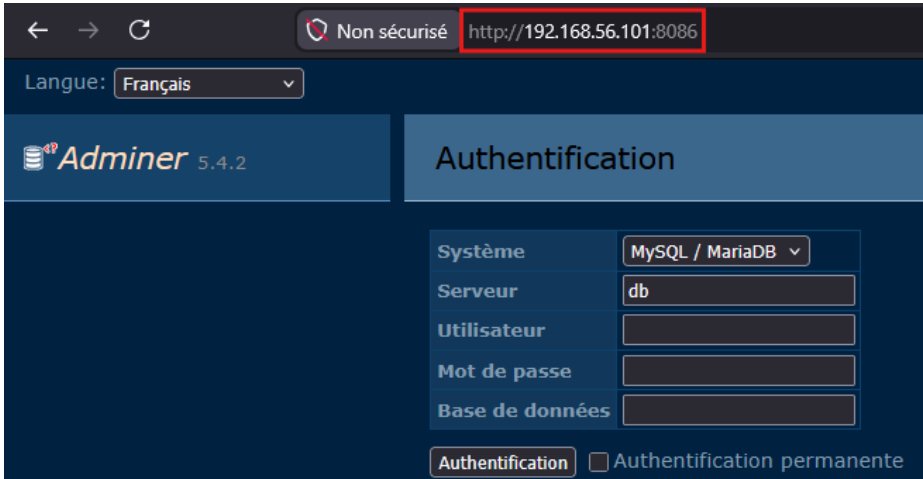
Etape	Description
1	Dans l'onglet "Stacks" sur Portainer, cliquer sur "Add stack".
	Représentation 
Etape	Description
2	Nous allons d'abord créer un serveur web simple. Il faut donc lui donner un nom, s'assurer que "web editor" est bien activé et entrer le code permettant la création du stack. Voici les explications du code: version: '3.8': version de syntaxe de <i>Docker Compose</i>. services: définit les différents services qui composent l'application. web: le nom du service. image: nginx:latest: indique à <i>Docker</i> de télécharger et d'utiliser l'image officielle NGINX avec le tag latest. container_name: nginx-web-simple: donne un nom spécifique au conteneur. ports: mappe le port interne du conteneur au port de l'hôte. Ici le port 80 de NGINX est accessible via le port 8085 de la machine hôte. restart: always: assure que le conteneur redémarre automatiquement en cas d'arrêt ou de redémarrage de Docker/la machine.

	Représentation Name nginx-simple This stack will be deployed using <code>docker compose</code>. Build method Web editor Upload Repository Custom template Web editor You can get more information about Compose file format in the official documentation. Define or paste the content of your docker compose file here <pre> 1 version: '3.8' 2 3 services: 4 web: 5 image: nginx:latest 6 container_name: nginx-web-simple 7 ports: 8 - "8085:80" 9 restart: always </pre>
Etape	Description
3	Déployer le stack, puis entrer l'adresse de la machine suivie du port, dans notre cas <code>192.168.56.101:8085</code>. Représentation

Etape	Description
4	Créer un nouveau répertoire pour y stocker le nouveau contenu avec mkdir -p /opt/nginx-content/html .
	Représentation
	<pre>root@srv-docker:~# mkdir -p /opt/nginx-content/html</pre>
Etape	Description
5	Créer le fichier <i>index.html</i> avec la commande nano /opt/nginx-content/html/index.html , puis y entrer le contenu souhaité.
	Représentation
	<pre>root@srv-docker:~# nano /opt/nginx-content/html/index.html <h1>Mon Serveur Web</h1> <p>Ce site est servi par NGINX via Docker et Portainer</p></pre>
Etape	Description
6	Sur <i>Portainer</i> , ajouter au code précédemment défini l'emplacement de la page html. Voici les explications du code: - avant les deux points: l'emplacement où se situe le site - après les deux points: l'emplacement par défaut. - ro (read only): signifie que l'on donne uniquement des droits en lecture
	Représentation
	<pre>version: '3.8' services: web: image: nginx:latest container_name: nginx-web-simple ports: - "8085:80" volumes: - /opt/nginx-content/html:/usr/share/nginx/html:ro restart: always</pre>

Etape	Description
7	Mettre à jour le stack.
	Représentation
	
Etape	Description
8	Vider le cache et rafraîchir la page pour voir apparaître les changements.
	Représentation
	
Etape	Description
9	Pour ajouter le sgbdr Mariadb à notre stack, il faut ajouter une partie au code défini précédemment.
	Voici les explications du code: - driver: bridge est un standard pour faire communiquer des conteneurs sur une même machine - volumes: db_data: on demande à Docker de réserver un espace de stockage persistant pour la base de données

	Représentation
	<pre> # Votre Base de Données db: image: mariadb:10.11 container_name: mariadb-db restart: always environment: MARIADB_ROOT_PASSWORD: mot_de_passe_robuste MARIADB_DATABASE: mon_site MARIADB_USER: utilisateur MARIADB_PASSWORD: mot_de_passe_utilisateur volumes: - db_data:/var/lib/mysql networks: - app-network networks: app-network: driver: bridge volumes: db_data: </pre>
Etape	Description
10	Pour gérer la base de données, nous allons utiliser <i>Adminer</i>. Il est gratuit et open-source et est contenu dans un seul fichier PHP. <i>Adminer</i> est encore plus léger que PHPMYAdmin. Pour cela, il faut donc ajouter un troisième container, toujours sur <i>Portainer</i>.
	Représentation
	<pre> adminer: image: adminer container_name: adminer restart: always ports: - 8086:8080 networks: - app-network networks: app-network: driver: bridge volumes: db_data: </pre>

Etape	Description
11	Dans le navigateur, entrer l'adresse IP suivie du port, dans notre cas <i>192.168.56.101:8086</i> , afin d'arriver sur l'interface d' <i>Adminer</i> .
	Représentation
	
Etape	Description
12	Se connecter avec les identifiants spécifiés lors de la création du conteneur. On accède alors à l'interface permettant de créer des tables.
	Représentation
	

4.7. Communication entre deux stacks

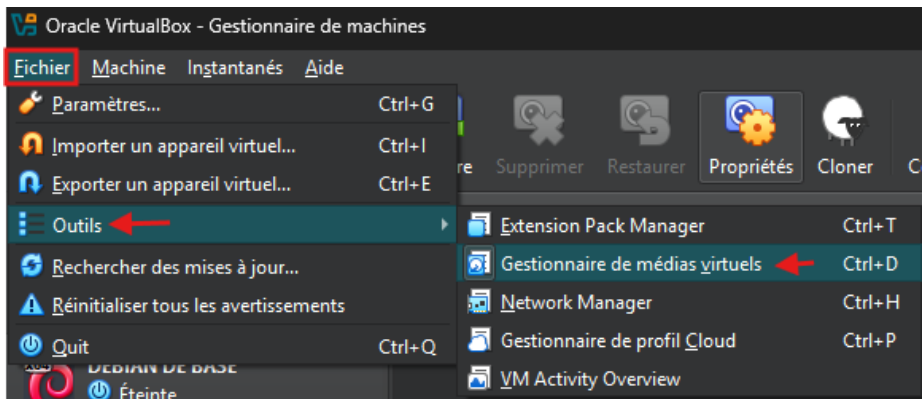
Dans cette section, nous allons faire communiquer deux stacks distincts: un stack *Jellyfin* (serveur multimédia) et un stack LDAP (gestion des comptes utilisateurs). L'objectif est de permettre à *Jellyfin* d'authentifier ses utilisateurs via un annuaire LDAP centralisé, démontrant ainsi la communication inter-stacks dans un environnement *Docker*.

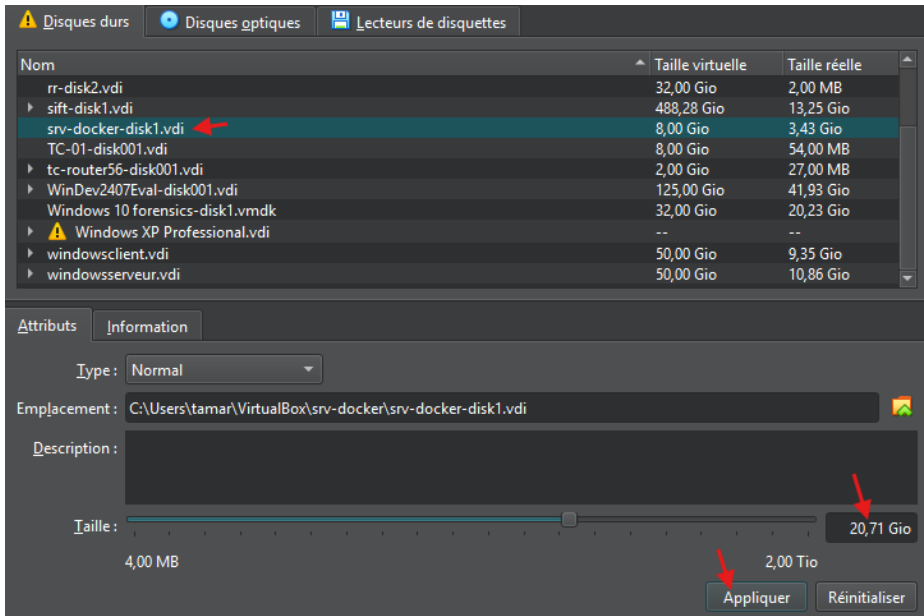
4.7.1. Mise en place de Jellyfin

Avec *Docker* et *Portainer*, déployer des stacks variés est facile. Dans le cadre d'une utilisation personnelle, un stack *Jellyfin* permet de centraliser films, séries, musiques et photos sur une seule machine. Sa mise en place nécessite de passer par quelques étapes.

4.7.1.1. Agrandissement du disque

Tout d'abord, le disque de base de la machine virtuelle n'est pas suffisant pour stocker des fichiers multimédias volumineux. Il est donc nécessaire de l'agrandir avant de déployer *Jellyfin*:

Etape	Description
1	Dans VirtualBox, sélectionner la Machine Virtuelle, puis dans l'onglet "Fichier", aller dans "Outils" et choisir "Gestionnaire de médias virtuels".
	Représentation
	

Etape	Description
2	Choisir le disque de la Machine Virtuelle et augmenter la taille du disque.
	Représentation
	 The screenshot shows the VirtualBox Disk Manager interface. At the top, there are tabs for 'Disques durs', 'Disques optiques', and 'Lecteurs de disquettes'. Below, a table lists several virtual disks. The disk 'srv-docker-disk1.vdi' is selected, highlighted in blue. A red arrow points to this row. Below the table, the 'Information' tab is active. It shows the disk type as 'Normal', the location as 'C:\Users\tamar\VirtualBox\srv-docker\srv-docker-disk1.vdi', and a size slider. The slider is set to 20,71 Gio, with a red arrow pointing to the value. At the bottom right, there are 'Appliquer' and 'Réinitialiser' buttons, with a red arrow pointing to 'Appliquer'.
Etape	Description
3	Désactiver le swap avec la commande swapoff -a .
	Représentation
	<pre>root@srv-docker:~# swapoff -a</pre>
Etape	Description
4	Vérifier la structure actuelle du disque avec fdisk /dev/sda .
	Représentation
	<pre>root@srv-docker:~# fdisk /dev/sda</pre>
Etape	Description
5	Afficher la table de partitions actuelle avec la commande p .

	Représentation <pre>Commande (m pour l'aide) : p Disque /dev/sda : 20,71 GiB, 22232760320 octets, 43423360 secteurs Modèle de disque : VBOX HARDDISK Unités : secteur de 1 × 512 = 512 octets Taille de secteur (logique / physique) : 512 octets / 512 octets taille d'E/S (minimale / optimale) : 512 octets / 512 octets Type d'étiquette de disque : dos Identifiant de disque : 0x7b743b31 Périphérique Amorçage Début Fin Secteurs Taille Id Type /dev/sda1 * 2048 14680063 14678016 7G 83 Linux /dev/sda2 14682110 16775167 2093058 1022M 5 Étendue /dev/sda5 14682112 16775167 2093056 1022M 82 partition d'échange Linux / Solaris</pre>
Etape	Description
6	Supprimer la partition de swap (partition 5) avec la commande d .
	Représentation
	<pre>Commande (m pour l'aide) : d Numéro de partition (1,2,5, 5 par défaut) : 5 La partition 5 a été supprimée.</pre>
Etape	Description
7	Supprimer la partition étendue (partition 2) avec la commande d .
	Représentation
	<pre>Commande (m pour l'aide) : d Numéro de partition (1,2, 2 par défaut) : 2 La partition 2 a été supprimée.</pre>
Etape	Description
8	Supprimer la partition principale (partition 1) avec la commande d .
	Représentation
	<pre>Commande (m pour l'aide) : d Partition 1 sélectionnée La partition 1 a été supprimée.</pre>

Etape	Description
9	Créer une nouvelle partition principale avec la commande n (nouvelle partition de type primaire, numéro 1, utilisant tout l'espace disponible).
	Représentation
	<pre>Sélectionnez (p par défaut) : p Numéro de partition (1-4, 1 par défaut) : 1 Premier secteur (2048-43423359, 2048 par défaut) : Dernier secteur, +/-secteurs ou +/-taille{K,M,G,T,P} (2048-43423359, 43423359 par défaut) : Une nouvelle partition 1 de type « Linux » et de taille 20,7 GiB a été créée.</pre>
Etape	Description
10	Répondre N pour conserver la signature ext4 existante.
	Représentation
	<pre>La partition #1 contient une signature ext4. Voulez-vous supprimer la signature ? [O]ui/[N]on : N</pre>
Etape	Description
11	Activer le flag de démarrage (bootable) sur la partition 1 avec la commande a .
	Représentation
	<pre>Commande (m pour l'aide) : a Partition 1 sélectionnée L'indicateur d'amorçage de la partition 1 est maintenant activé.</pre>
Etape	Description
12	Écrire les modifications sur le disque avec la commande w .
	Représentation
	<pre>Commande (m pour l'aide) : w La table de partitions a été altérée. Synchronisation des disques.</pre>

Etape	Description
13	Informez le noyau des nouvelles partitions sans redémarrage avec partx -u /dev/sda1 .
	Représentation
	<pre>root@srv-docker:~# partx -u /dev/sda1</pre>
Etape	Description
14	Redimensionner le système de fichiers pour utiliser tout l'espace disponible avec resize2fs /dev/sda1 .
	Représentation
	<pre>root@srv-docker:~# resize2fs /dev/sda1 resize2fs 1.46.2 (28-Feb-2021) Filesystem at /dev/sda1 is mounted on /; on-line resizing required old_desc_blocks = 1, new_desc_blocks = 1 The filesystem on /dev/sda1 is now 2096896 (4k) blocks long.</pre>
Etape	Description
15	Vérifier le résultat avec df -h / pour confirmer la nouvelle taille du disque.
	Représentation
	<pre>root@srv-docker:~# df -h Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur udev 465M 0 465M 0% /dev tmpfs 97M 624K 96M 1% /run /dev/sda1 21G 2,3G 18G 12% /</pre> 

4.7.1.2. Préparation des répertoires

Ensuite, il est nécessaire de créer les répertoires sur la machine et notamment le répertoire qui accueillera les médias:

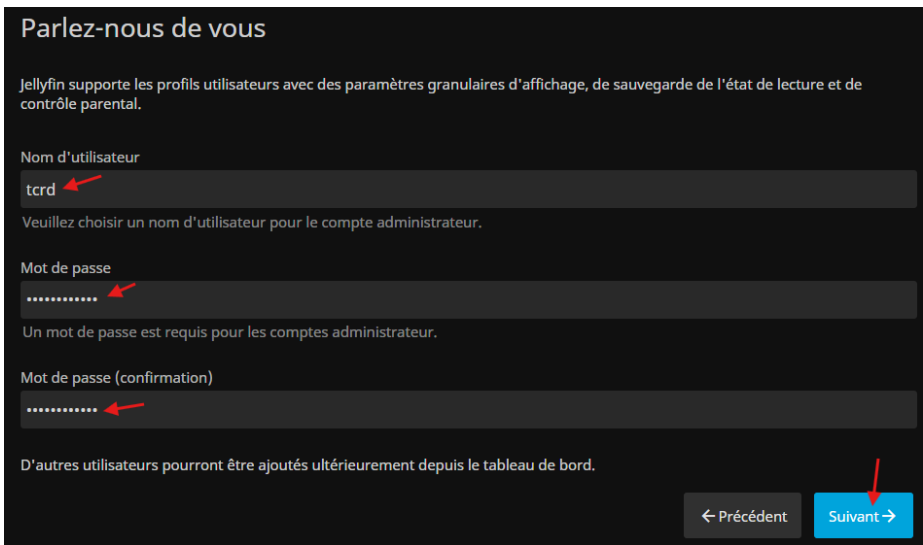
```
root@srv-docker:~# mkdir -p /home/sio/jellyfin/config
root@srv-docker:~# mkdir -p /home/sio/jellyfin/cache
root@srv-docker:~# mkdir -p /home/sio/media
```

4.7.1.3. Déploiement du stack Jellyfin

Enfin, il faut passer au déploiement du stack *Jellyfin* qui nécessite également de passer par plusieurs étapes:

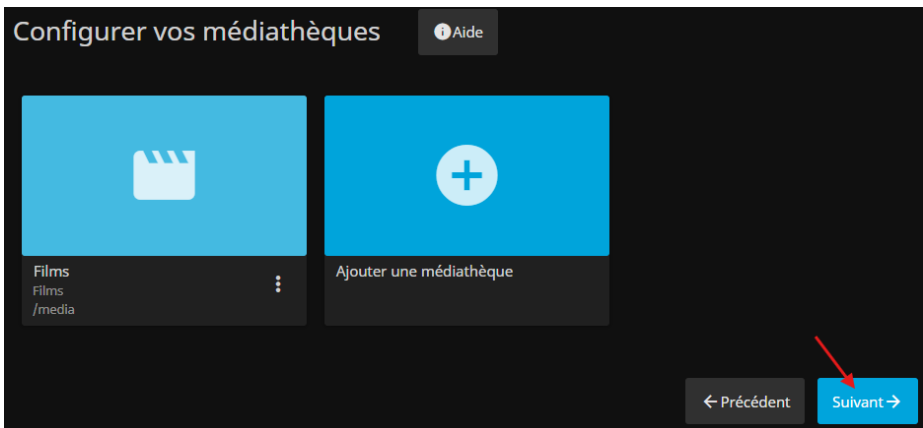
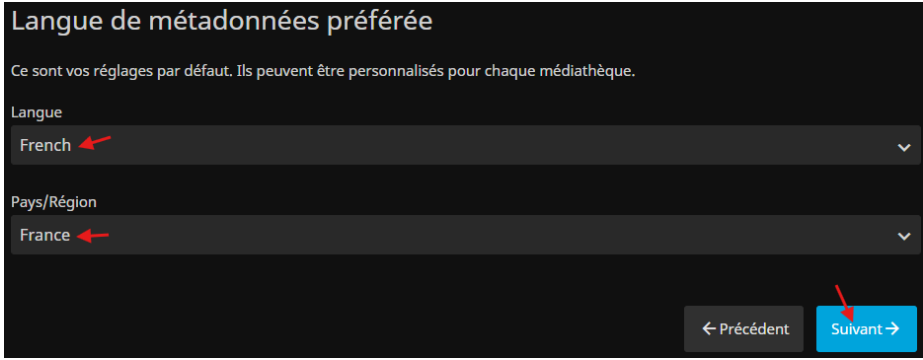
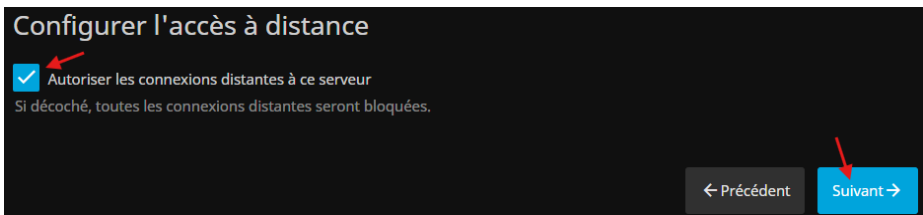
Etape	Description
1	Entrer la commande id sio pour récupérer l'ID de l'utilisateur "sio".
	Représentation
	<pre>root@srv-docker:~# id sio uid=1000(sio) gid=1000(sio) groupes=1000(sio),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),109(netdev)</pre>
Etape	Description
2	Modifier les propriétés des répertoires pour l'utilisateur "sio" avec les commandes chown -R 1000:1000 /home/sio/jellyfin et chown -R 1000:1000 /home/sio/media .
	Représentation
	<pre>root@srv-docker:~# chown -R 1000:1000 /home/sio/jellyfin root@srv-docker:~# chown -R 1000:1000 /home/sio/media</pre>
Etape	Description
3	Ensuite, dans <i>Portainer</i> , créer un stack, puis entrer le code suivant dans Web editor, après donné un nom au stack.
	Représentation
	<pre>services: jellyfin: image: jellyfin/jellyfin container_name: jellyfin user: 1000:1000 # Utilisateur SIO ports: - 8096:8096/tcp - 7359:7359/udp volumes: - /home/sio/jellyfin/config:/config - /home/sio/jellyfin/cache:/cache - /home/sio/media:/media restart: unless-stopped environment: - JELLYFIN_PublishedServerUrl=http://192.168.56.101 extra_hosts:</pre>

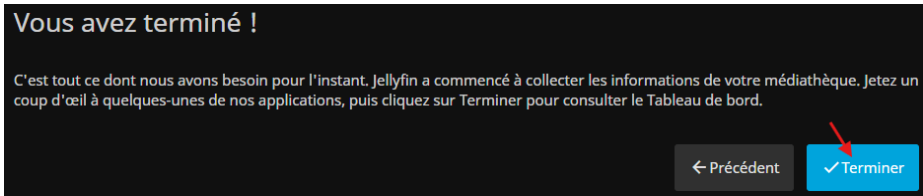
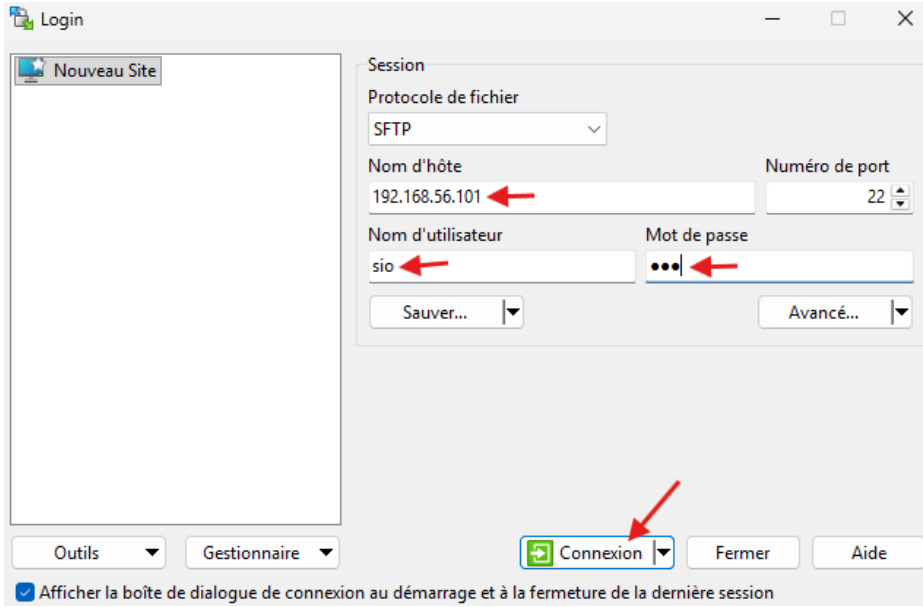
	- 'host.docker.internal:host-gateway' Name* jellyfin This stack will be deployed using <code>docker compose</code>. Build method Web editor Upload Repository Custom template Web editor Ctrl+F for search Define or paste the content of your docker compose file here Copy 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 services: jellyfin: image: jellyfin/jellyfin container_name: jellyfin user: 1000:1000 # Utilisateur SIO ports: - 8096:8096/tcp - 7359:7359/udp volumes: - /home/sio/jellyfin/config:/config - /home/sio/jellyfin/cache:/cache - /home/sio/media:/media restart: unless-stopped environment: - JELLYFIN_PublishedServerUrl=http://192.168.56.101 extra_hosts: - 'host.docker.internal:host-gateway'
Etape	Description
4	Déployer le stack, puis se rendre à l'adresse du conteneur dans un navigateur. Donner un nom au serveur, choisir la langue d'affichage et cliquer sur "Suivant".
	Représentation ← → ↻ Non sécurisé http://192.168.56.101:8096 web/#/wizard/start 🔍 ⚙️ ☆ Jellyfin Bienvenue dans Jellyfin ! Guide de démarrage rapide Cet assistant vous guidera dans le processus de configuration. Pour commencer, merci d'entrer votre nom et de sélectionner votre langue préférée. Nom du serveur multimédia Ce nom sera utilisé pour identifier le serveur. La valeur par défaut est le nom d'hôte du serveur. Langue d'affichage préférée Français Suivant →

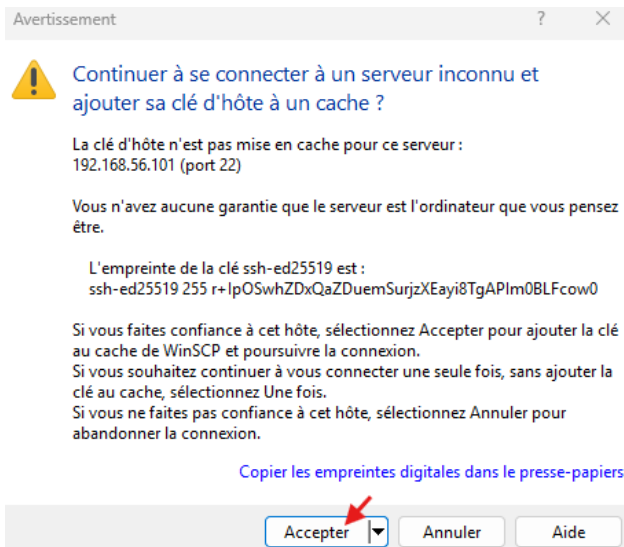
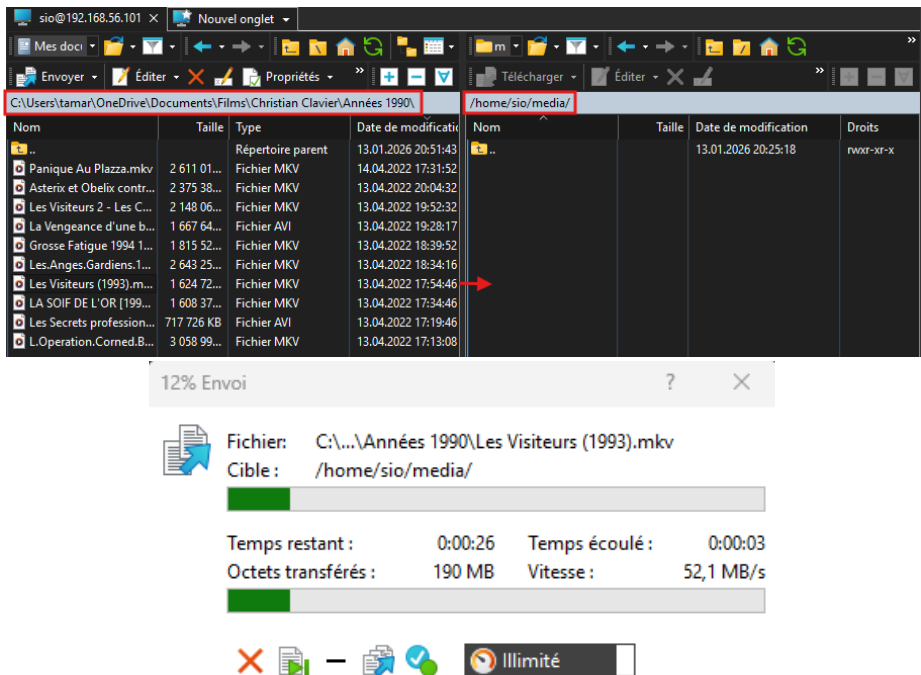
Etape	Description
5	Créer un utilisateur avec un mot de passe et cliquer sur “Suivant”.
	Représentation
	
Etape	Description
6	Cliquer sur “Ajouter une médiathèque”.
	Représentation
	
Etape	Description
7	Sélectionner le type de contenu, dans notre cas ce seront des films. Donner également un nom d’affichage au contenu.

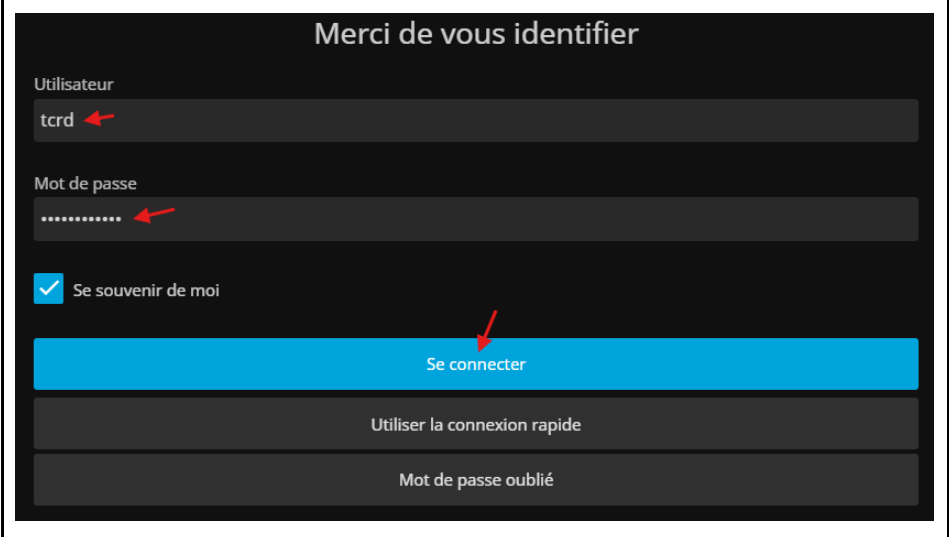
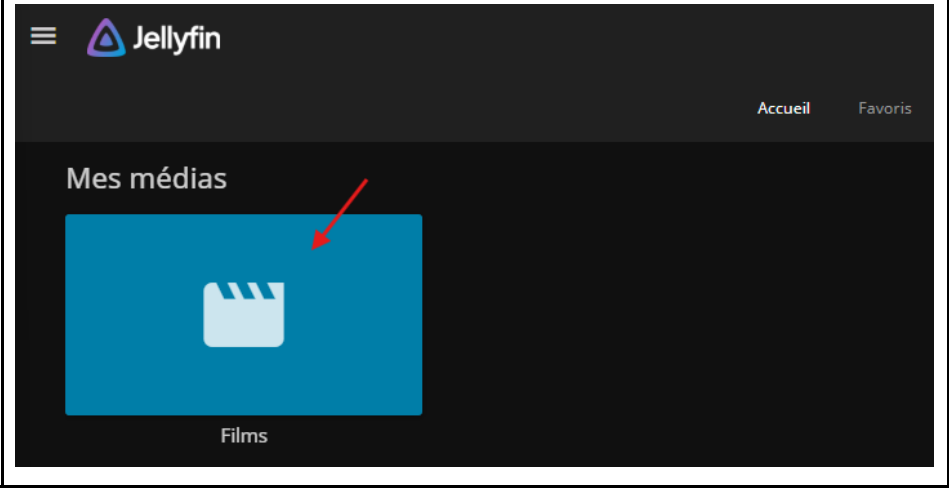
	Représentation Type de contenu Nom d'affichage Dossiers + Type de contenu Films Musiques Séries Livres Vidéos et photos personnelles Clips musicaux Films et séries mélangés Type de contenu Films Consultez le guide de nommage des films. Nom d'affichage Films
Etape	Description
8	Cliquer sur le + et entrer le lien vers le répertoire où se trouveront les films. Cliquer ensuite sur “OK”. Représentation Dossiers + Dossier /media ... OK

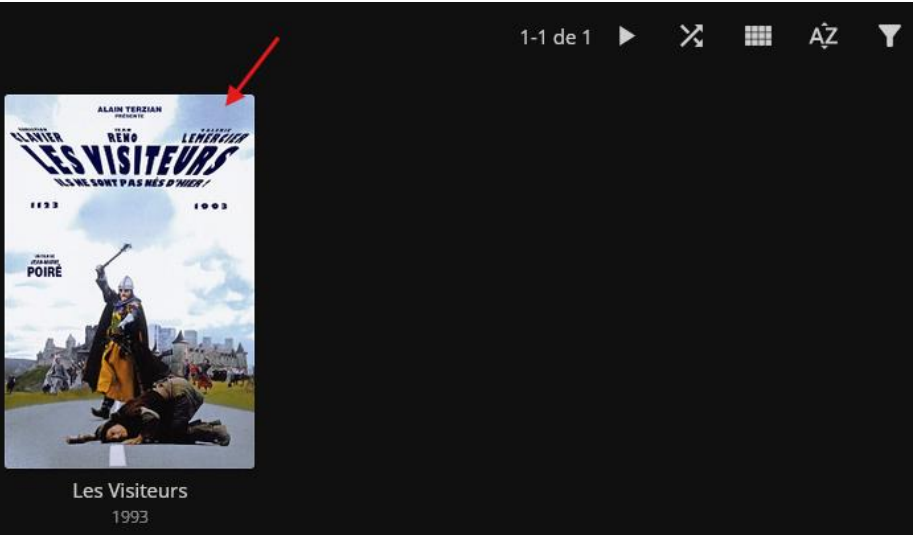
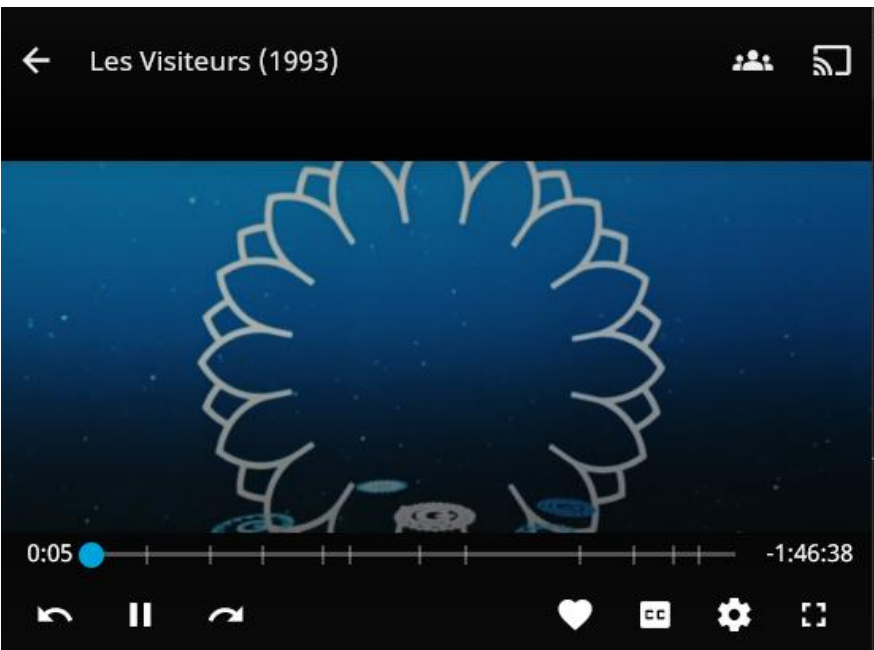
Etape	Description
9	Choisir la langue préférée et cliquer sur “OK”. Il est possible d’activer des options si souhaité.
	Représentation
	
Etape	Description
10	Une fois la médiathèque créée, cliquer sur “Suivant”.

	Représentation 
Etape	Description
11	Choisir la langue de métadonnées préférée et cliquer sur "Suivant". Représentation 
Etape	Description
12	Configurer l'accès à distance en cochant "Autoriser les connexions distantes à ce serveur", puis cliquer sur "Suivant". Représentation 

Etape	Description
13	Cliquer sur "Terminer" pour finaliser la configuration.
	Représentation
	
Etape	Description
14	Se connecter via WinSCP pour transférer des fichiers multimédias dans le répertoire /home/sio/media. Pour cela, choisir le protocole SFTP, le port 22, entrer l'adresse IP de la Machine Virtuelle et les identifiants de connexion à la machine pour l'utilisateur "sio".
	Représentation
	
Etape	Description
15	Accepter la clé de l'hôte inconnu en cliquant sur "Accepter".

	Représentation 
Etape	Description
16	Une fois connecté, transférer les fichiers multimédias vers le répertoire /home/sio/media. Représentation 
Etape	Description
17	Se connecter à Jellyfin avec les identifiants créés précédemment.

	Représentation 
Etape	Description
18	Cliquez sur la médiathèque "Films". Représentation 

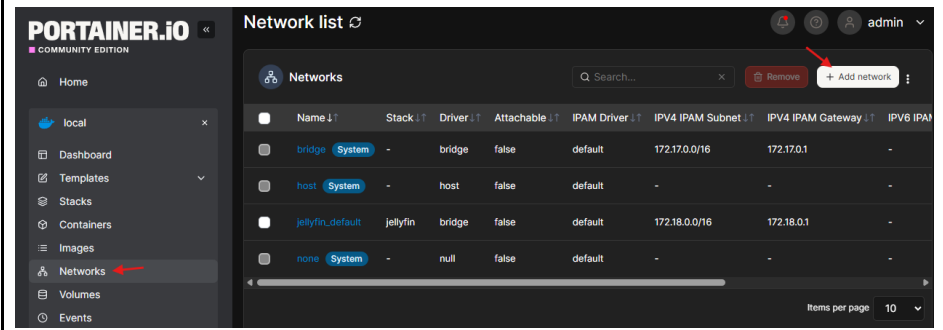
Etape	Description
19	Le film transféré grâce à WinSCP s'y trouve bien. Cliquer dessus.
	Représentation
	
Etape	Description
20	Le film se lance alors.
	Représentation
	

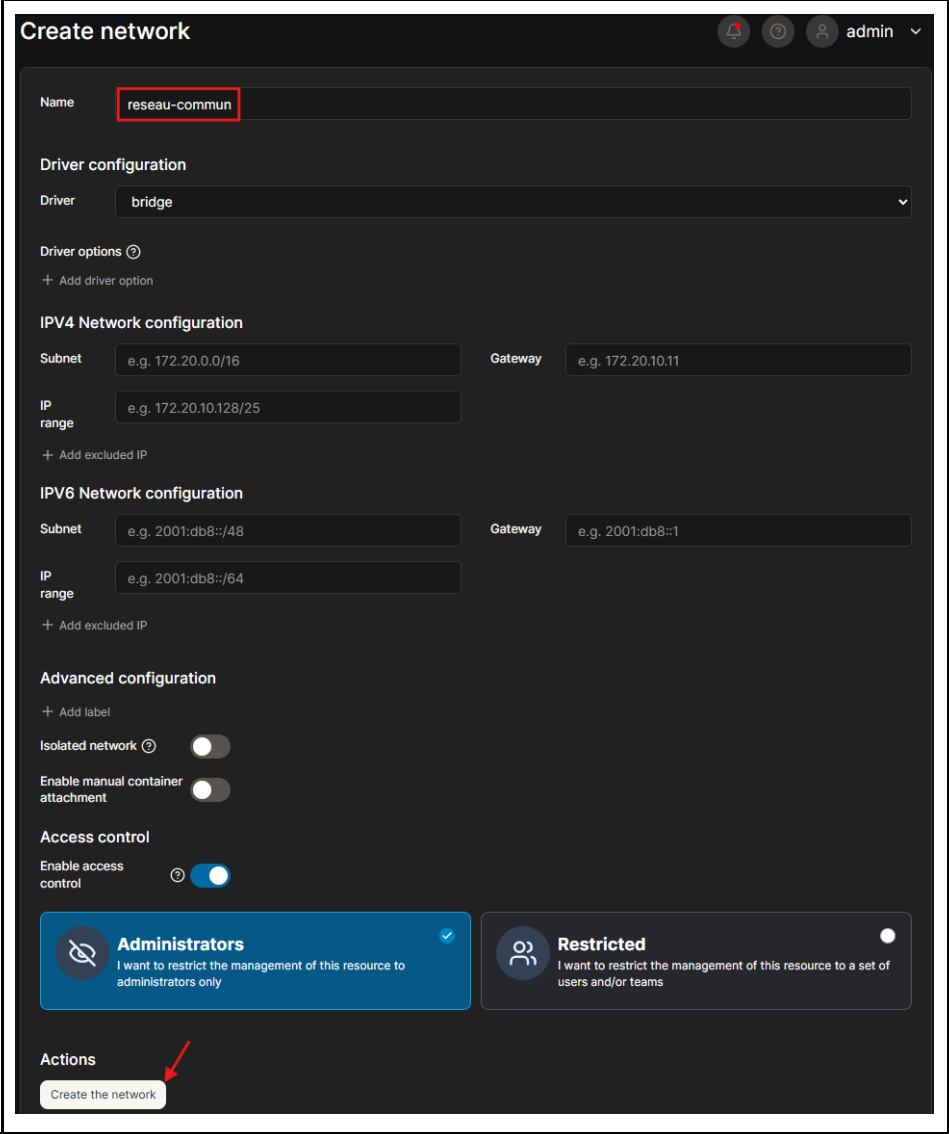
4.7.2. Déploiement du stack LDAP

Pour permettre la communication entre les deux stacks, nous allons maintenant déployer un stack LDAP composé de deux conteneurs: *OpenLDAP* (annuaire d'utilisateurs) et *phpLDAPadmin* (interface web de gestion). Ce stack va partager un réseau commun avec le stack *Jellyfin* pour permettre l'authentification centralisée.

4.7.2.1. Création du réseau commun

Avant de déployer les stacks, il est nécessaire de créer un réseau Docker commun qui permettra aux conteneurs des deux stacks de communiquer entre eux:

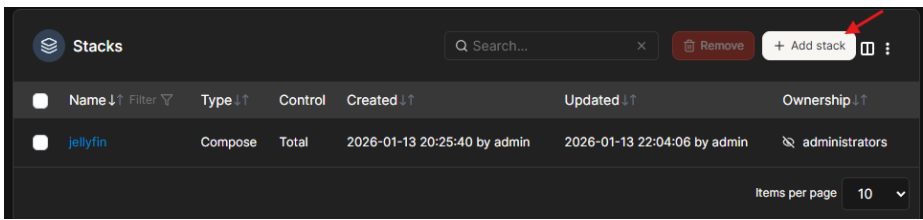
Etape	Description
1	Dans Portainer, aller dans l'onglet "Networks" puis cliquer sur "Add network".
	Représentation 

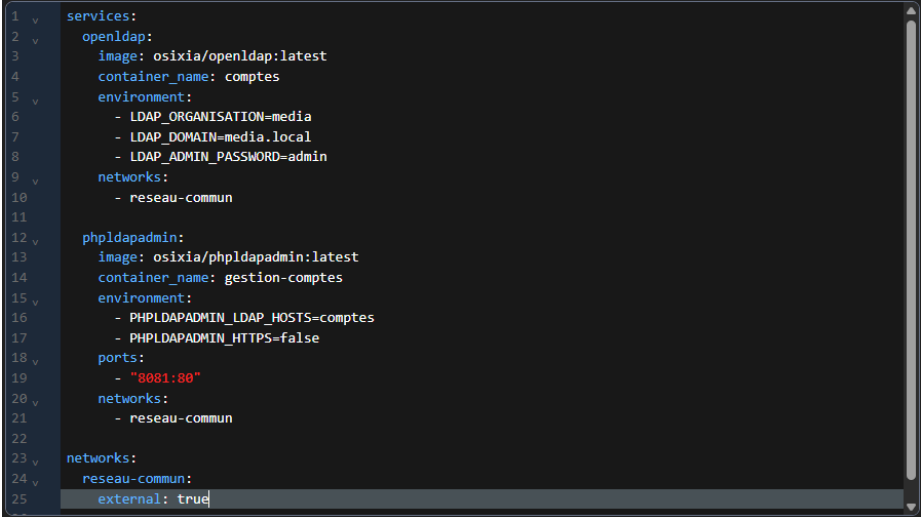
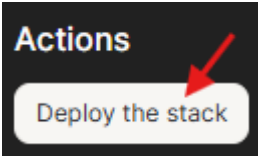
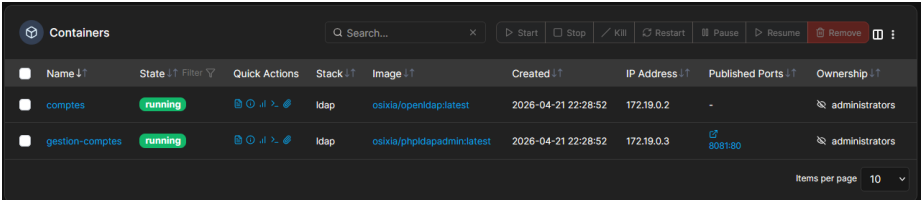
Etape	Description
2	Nommer le réseau "reseau-commun", laisser le driver sur "bridge", puis cliquer sur "Create the network".
	Représentation
	

4.7.2.2. Déploiement du stack LDAP

Maintenant que le réseau commun est créé, nous pouvons déployer le stack *LDAP*:

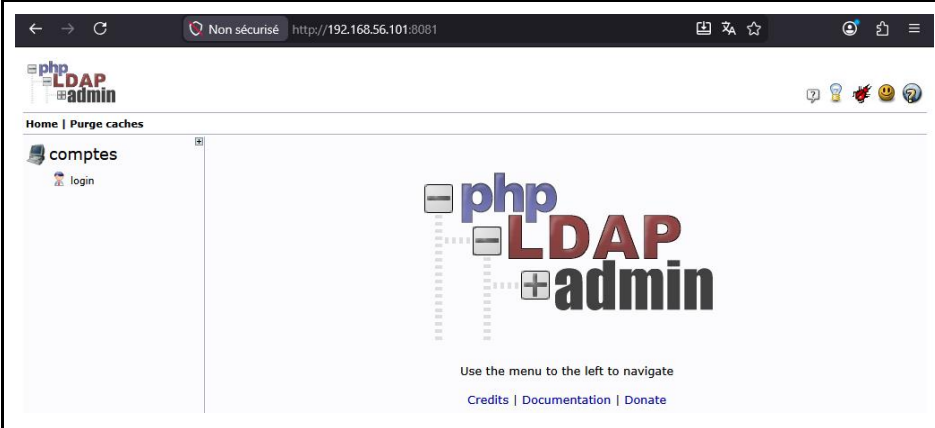
Etape	Description
1	Dans l'onglet "Stacks" sur Portainer, cliquer sur "Add stack"

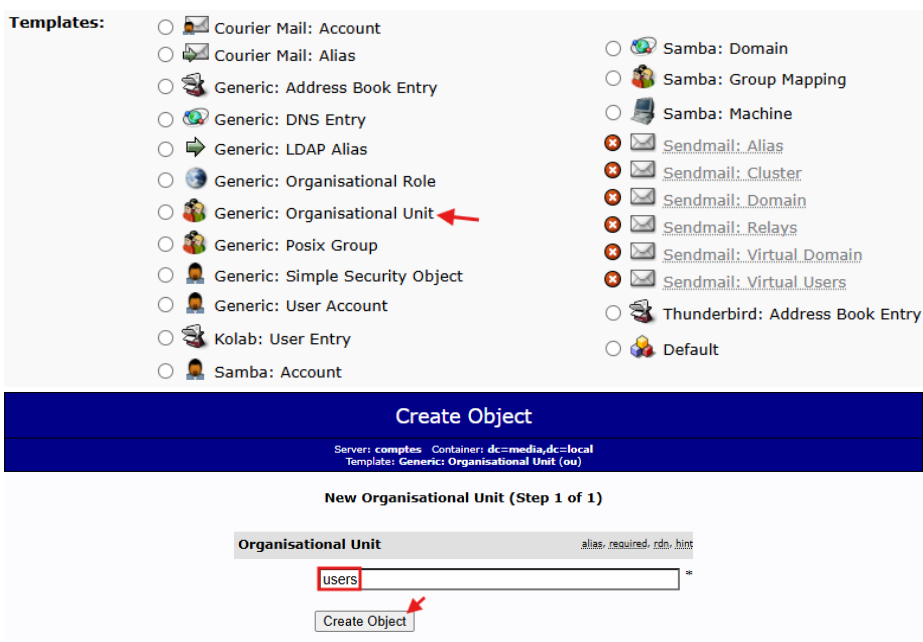
	Représentation 
Etape	Description
2	Donner le nom "ldap" au stack, puis entrer le code suivant dans le Web editor: <pre> services: openldap: image: osixia/openldap:latest container_name: comptes environment: - LDAP_ORGANISATION=media - LDAP_DOMAIN=media.local - LDAP_ADMIN_PASSWORD=admin network: - reseau-commun networks: reseau-commun: external: true </pre>

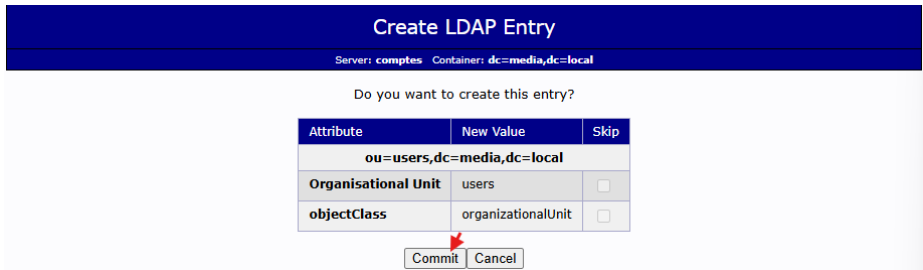
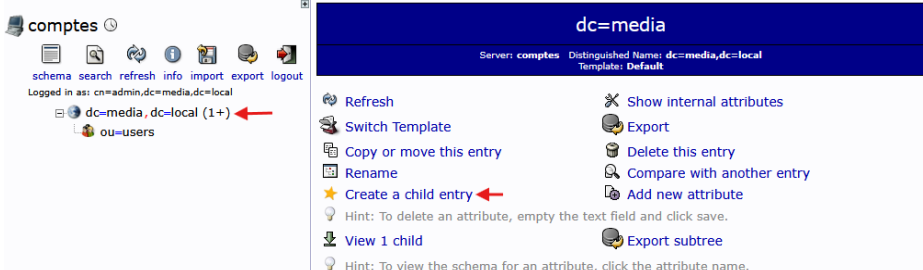
	Représentation 
Etape	Description
3	Déployer le stack en cliquant sur "Deploy the stack".
	Représentation 
Etape	Description
4	Vérifier que les conteneurs “comptes” et “gestion-comptes” sont bien en cours d'exécution dans la liste des conteneurs.
	Représentation 

4.7.2.3. Création des utilisateurs dans phpLDAPadmin

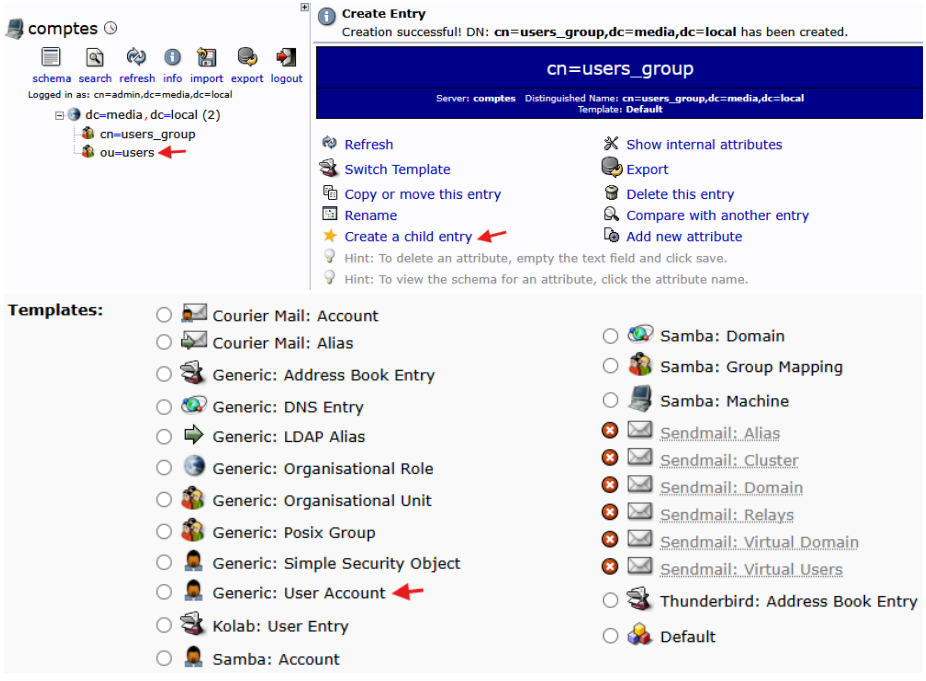
Une fois le stack LDAP déployé, nous allons utiliser l'interface *phpLDAPadmin* pour créer des utilisateurs qui pourront ensuite s'authentifier sur *Jellyfin*:

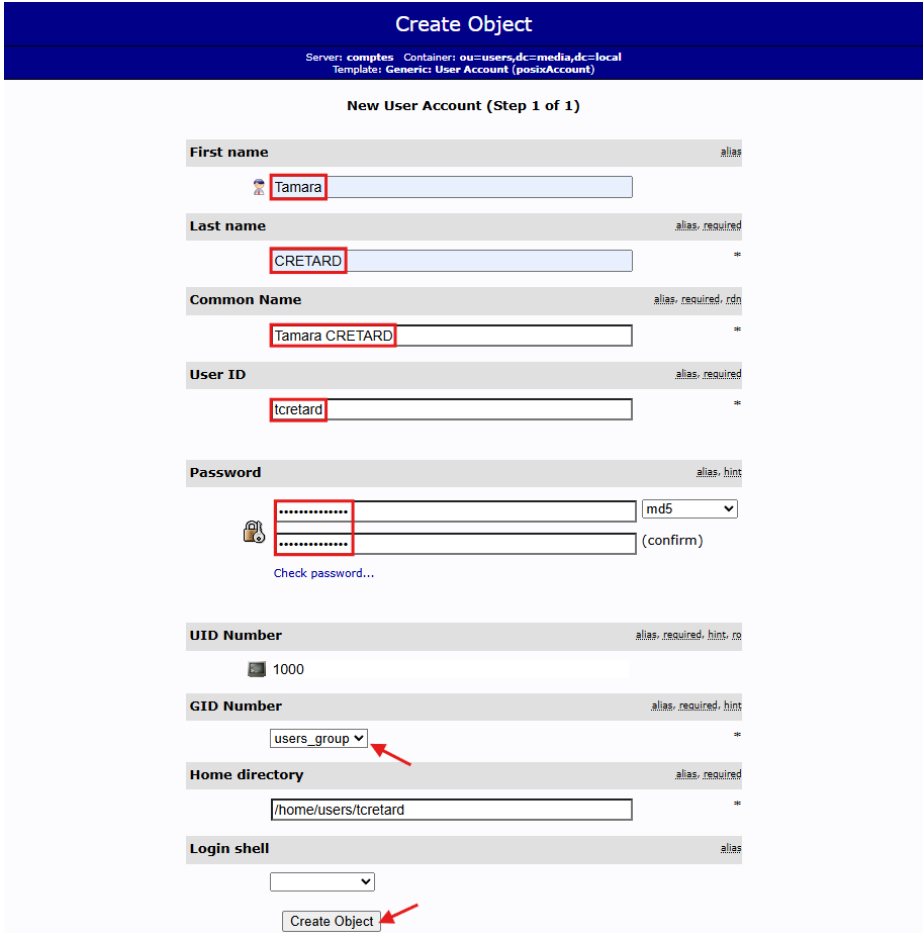
Etape	Description
1	Accéder à <i>phpLDAPadmin</i> dans un navigateur à l'adresse de la machine suivie du numéro de port, dans notre cas, 192.168.56.101:8081.
	Représentation
	
Etape	Description
2	Se connecter avec le Login DN: cn=admin,dc=media,dc=local et le mot de passe admin.
	Représentation
	
Etape	Description
3	Une fois connecté, cliquer sur dc=media,dc=local dans le panneau de gauche, puis sur "Create a child entry".

	Représentation 
Etape	Description
4	Sélectionner "Generic: Organisational Unit" et créer une unité organisationnelle nommée "users". Représentation 

Etape	Description
5	Confirmer la création de l'unité organisationnelle ou=users,dc=media,dc=local en cliquant sur "Commit".
	Représentation
	
Etape	Description
6	Créer ensuite un groupe en cliquant sur dc=media,dc=local, puis "Create a child entry".
	Représentation
	
Etape	Description
7	Choisir "Generic: Posix Group" et créer un groupe nommé "users_group" avec le GID 500.

	Représentation Templates: ☐  Courier Mail: Account ☐  Courier Mail: Alias ☐  Generic: Address Book Entry ☐  Generic: DNS Entry ☐  Generic: LDAP Alias ☐  Generic: Organisational Role ☐  Generic: Organisational Unit ☐  Generic: Posix Group  ☐  Generic: Simple Security Object ☐  Generic: User Account ☐  Kolab: User Entry ☐  Samba: Account ☐  Samba: Domain ☐  Samba: Group Mapping ☐  Samba: Machine ☒  Sendmail: Alias ☒  Sendmail: Cluster ☒  Sendmail: Domain ☒  Sendmail: Relays ☒  Sendmail: Virtual Domain ☒  Sendmail: Virtual Users ☐  Thunderbird: Address Book Entry ☐  Default Create Object Server: comptes Container: dc=media,dc=local Template: Generic: Posix Group (posixGroup) New Posix Group (Step 1 of 1) Group alias, required, rdn users_group GID Number alias, required, hint, ro 500 Users alias, hint Create Object 														
Etape	Description														
8	Confirmer la création du groupe.														
	Représentation Create LDAP Entry Server: comptes Container: dc=media,dc=local Do you want to create this entry? <table><tr><th>Attribute</th><th>New Value</th><th>Skip</th></tr><tr><td colspan="3">cn=users_group,dc=media,dc=local</td></tr><tr><td>Group</td><td>users_group</td><td>☐</td></tr><tr><td>GID Number</td><td>500</td><td>☐</td></tr><tr><td>objectClass</td><td>posixGroup</td><td>☐</td></tr></table> Create  Cancel	Attribute	New Value	Skip	cn=users_group,dc=media,dc=local			Group	users_group	☐	GID Number	500	☐	objectClass	posixGroup
Attribute	New Value	Skip													
cn=users_group,dc=media,dc=local															
Group	users_group	☐													
GID Number	500	☐													
objectClass	posixGroup	☐													

Etape	Description
9	Cliquer sur "ou=users" dans le panneau de gauche, puis "Create a child entry", choisir "Generic: User Account" pour créer un utilisateur.
	Représentation
	 The screenshot displays the LDAP browser interface. On the left, the 'comptes' tree shows the hierarchy: 'dc=media, dc=local' containing 'cn=users_group' and 'ou=users' (highlighted with a red arrow). The right pane shows the 'Create Entry' dialog. The title bar indicates 'cn=users_group'. Below, it shows 'Server: comptes' and 'Distinguished Name: cn=users_group,dc=media,dc=local'. A list of actions is provided, with 'Create a child entry' highlighted by a red arrow. At the bottom, a 'Templates:' section lists various entry types, with 'Generic: User Account' highlighted by a red arrow.

Etape	Description
10	Remplir le formulaire de création d'utilisateur en indiquant notamment le nom, prénom, ID et mot de passe de l'utilisateur. Il est aussi nécessaire de sélectionner "users_group" pour le GID Number.
	Représentation
	 The screenshot shows the 'Create Object' form for creating a new user account. The form is titled 'New User Account (Step 1 of 1)'. It contains the following fields: First name: Tamara Last name: CRETARD Common Name: Tamara CRETARD User ID: tcretard Password: md5 (with a confirmation field) UID Number: 1000 GID Number: users_group (selected from a dropdown menu) Home directory: /home/users/tcretard Login shell: (empty dropdown menu) Red boxes highlight the input fields for First name, Last name, Common Name, User ID, Password, and GID Number. Red arrows point to the 'users_group' dropdown and the 'Create Object' button.

Etape	Description																																
11	Confirmer la création de l'utilisateur en cliquant sur "Commit".																																
	Représentation																																
	Create LDAP Entry Server: comptes Container: ou=users,dc=media,dc=local Do you want to create this entry? <table><thead><tr><th>Attribute</th><th>New Value</th><th>Skip</th></tr></thead><tbody><tr><td colspan="3">cn=Tamara CRETARD,ou=users,dc=media,dc=local</td></tr><tr><td>First name</td><td>Tamara</td><td>☐</td></tr><tr><td>Last name</td><td>CRETARD</td><td>☐</td></tr><tr><td>Common Name</td><td>Tamara CRETARD</td><td>☐</td></tr><tr><td>User ID</td><td>tcretard</td><td>☐</td></tr><tr><td>Password</td><td>*****</td><td>☐</td></tr><tr><td>UID Number</td><td>1000</td><td>☐</td></tr><tr><td>GID Number</td><td>500</td><td>☐</td></tr><tr><td>Home directory</td><td>/home/users/tcretard</td><td>☐</td></tr><tr><td>objectClass</td><td>inetOrgPerson posixAccount</td><td>☐</td></tr></tbody></table>  Commit Cancel	Attribute	New Value	Skip	cn=Tamara CRETARD,ou=users,dc=media,dc=local			First name	Tamara	☐	Last name	CRETARD	☐	Common Name	Tamara CRETARD	☐	User ID	tcretard	☐	Password	*****	☐	UID Number	1000	☐	GID Number	500	☐	Home directory	/home/users/tcretard	☐	objectClass	inetOrgPerson posixAccount
Attribute	New Value	Skip																															
cn=Tamara CRETARD,ou=users,dc=media,dc=local																																	
First name	Tamara	☐																															
Last name	CRETARD	☐																															
Common Name	Tamara CRETARD	☐																															
User ID	tcretard	☐																															
Password	*****	☐																															
UID Number	1000	☐																															
GID Number	500	☐																															
Home directory	/home/users/tcretard	☐																															
objectClass	inetOrgPerson posixAccount	☐																															

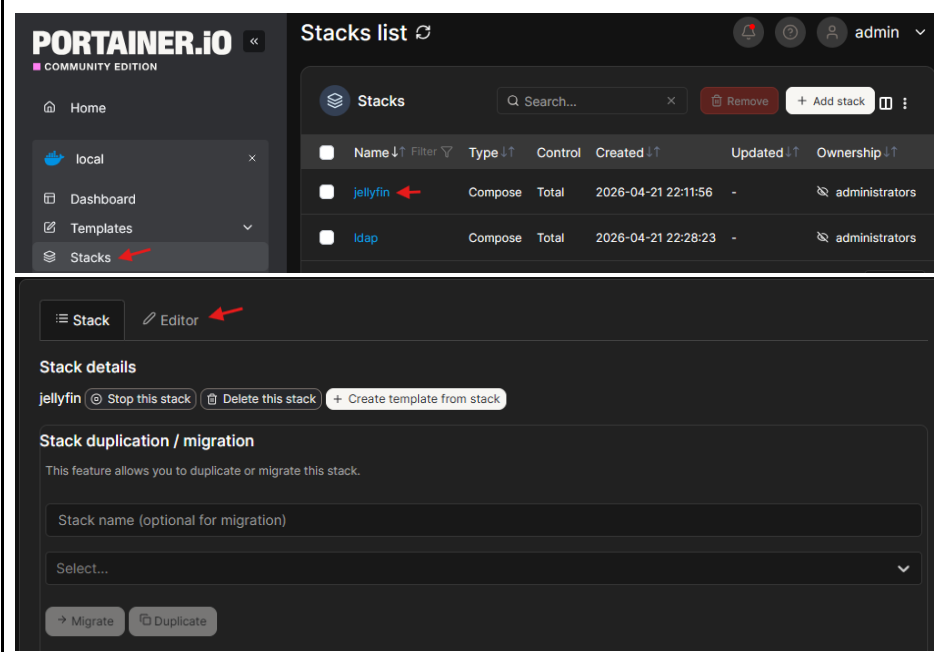
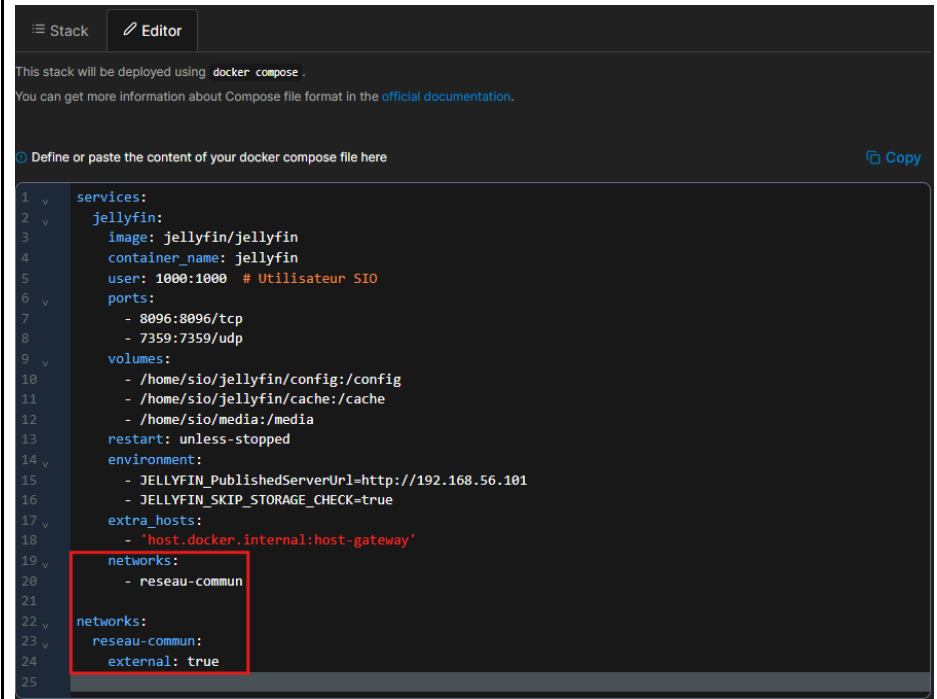
4.7.3. Connexion LDAP - Jellyfin

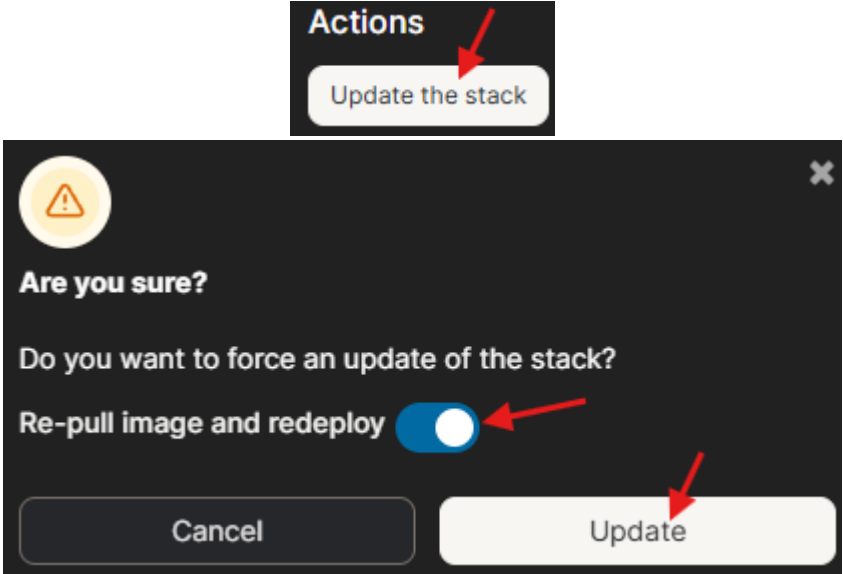
Maintenant que les deux stacks sont déployés et partagent le même réseau commun, nous allons configurer *Jellyfin* pour qu'il utilise l'annuaire LDAP pour authentifier les utilisateurs.

4.7.3.1. Mise à jour du stack Jellyfin

Pour que *Jellyfin* puisse communiquer avec le conteneur LDAP, il faut d'abord l'ajouter au réseau commun en modifiant le stack:

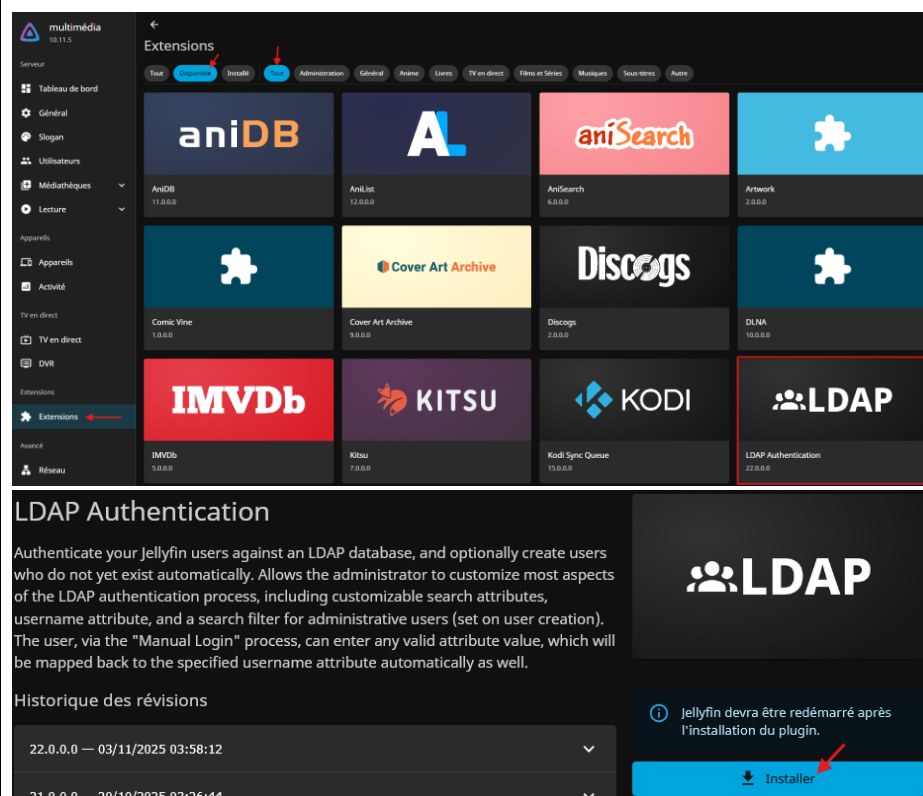
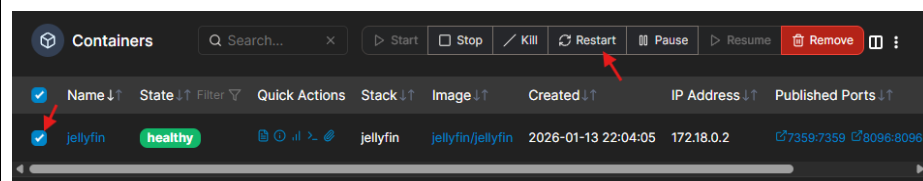
Etape	Description
1	Dans Portainer, aller dans l'onglet "Stacks", cliquer sur le stack "jellyfin", puis sur l'onglet "Editor".

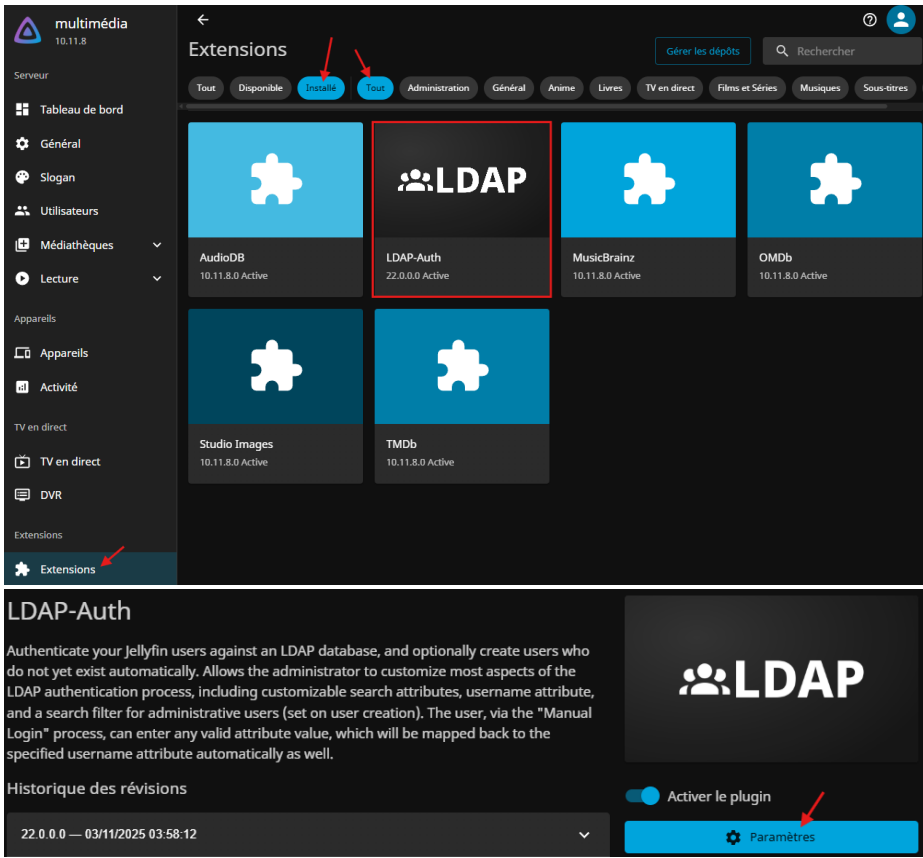
	Représentation 
Etape	Description
2	Modifier le code du stack Jellyfin pour y ajouter le réseau commun. Représentation 

Etape	Description
3	Mettre à jour le stack en cliquant sur "Update the stack" et confirmer avec "Update".
	Représentation
	

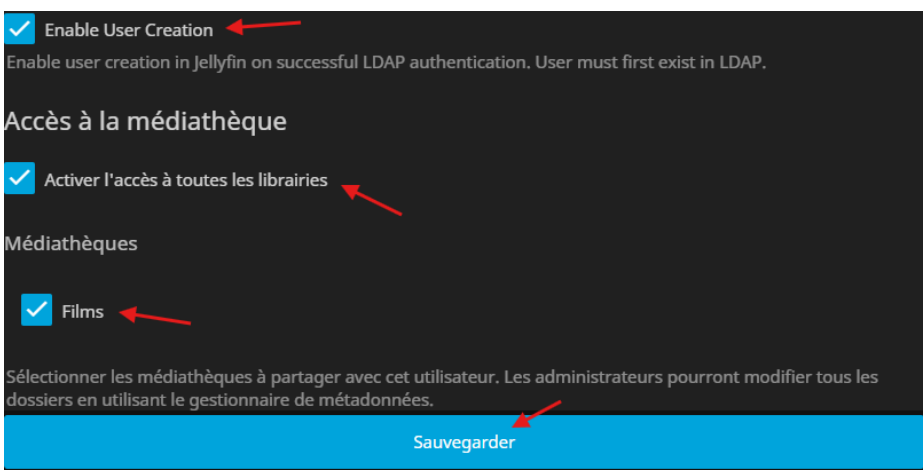
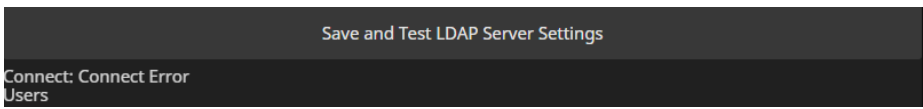
4.7.3.2. Installation et configuration du plugin LDAP dans Jellyfin

Une fois le réseau commun configuré, il faut installer le plugin LDAP dans *Jellyfin* et le paramétrer pour qu'il se connecte au conteneur OpenLDAP:

Etape	Description
1	Dans l'interface Jellyfin, aller dans "Administration", "Tableau de bord", "Extensions", puis sélectionner "Disponible". Chercher l'extension "LDAP Authentication" et cliquer sur "Installer".
	Représentation 
Etape	Description
2	Se rendre dans l'onglet "Containers" de Portainer et redémarrer <i>Jellyfin</i> pour que le plugin soit pris en compte.
	Représentation 

Etape	Description
3	Revenir dans “Extensions” et cliquer sur l'onglet "Installé". Cliquer ensuite sur “Paramètres”.
	Représentation  The screenshot shows the Jellyfin web interface. On the left is a sidebar with navigation options like 'Tableau de bord', 'Général', 'Slogan', 'Utilisateurs', 'Médiathèques', 'Lecture', 'Appareils', 'Activité', 'TV en direct', 'DVR', and 'Extensions'. The 'Extensions' menu item is highlighted with a red arrow. The main area displays the 'Extensions' page with tabs for 'Tout', 'Disponible', 'Installé', and 'Tout'. The 'Installé' tab is selected, and the 'LDAP' extension is highlighted with a red box. Below the extension list, the 'LDAP-Auth' configuration page is shown, featuring a description, a version history section, and a toggle switch for 'Activer le plugin' which is turned on. A red arrow points to the 'Paramètres' button at the bottom right of the configuration page.
Etape	Description
4	Configurer les paramètres LDAP: - LDAP Server: comptes (nom du conteneur OpenLDAP) - LDAP Port: 389 - Décocher Secure LDAP et StartTLS - LDAP Bind User: CN=admin,DC=media,DC=local - LDAP Bind User Password: admin - LDAP Base DN for searches: ou=users,dc=media,dc=local

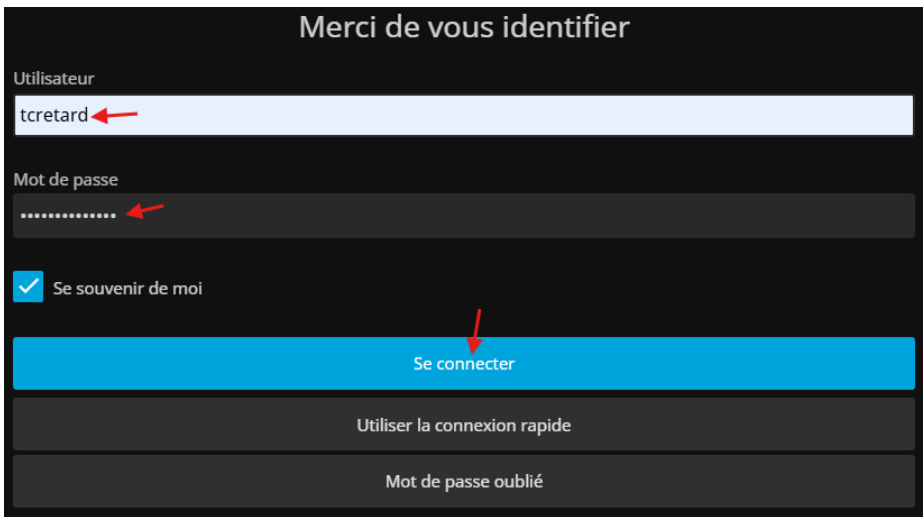
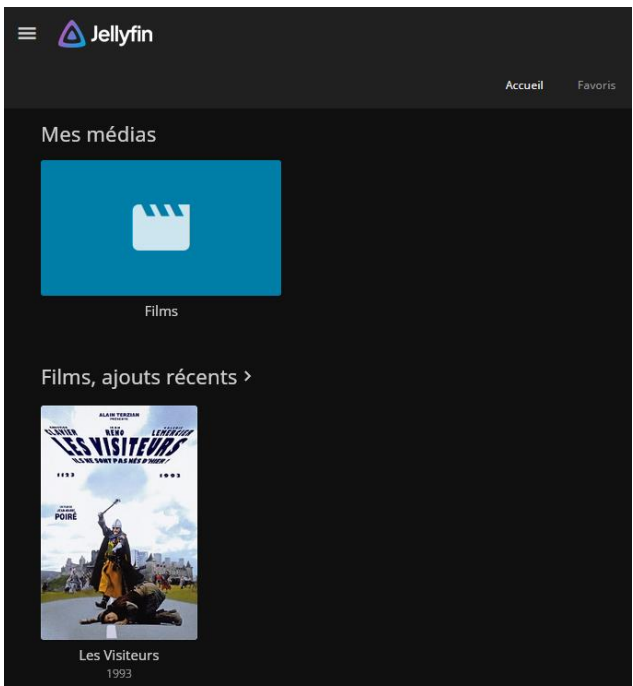
	Représentation LDAP Server: comptes The server name for the LDAP server you wish to use for Authentication. LDAP Port: 389 ☐ Secure LDAP Use SSL for the LDAP connection. ☐ StartTLS Use StartTLS for the LDAP connection. LDAP Bind User: CN=admin,DC=media,DC=local A bind user for LDAP search queries, required if your LDAP doesn't support anonymous bind. LDAP Bind User Password: ***** The password for the LDAP search query user. LDAP Base DN for searches: ou=users,dc=media,dc=local
Etape	Description
5	Configurer les attributs de recherche: - LDAP Search Attributes: uid - LDAP Uid Attribute: uid - LDAP Username Attribute: uid - Cocher "Enable User Creation" - Cocher "Activer l'accès à toutes les librairies" et sélectionner la médiathèque "Films" Puis cliquer sur "Sauvegarder". Représentation LDAP Search Attributes: uid A comma-separated list of attributes to search for the username. LDAP Uid Attribute: uid The LDAP attribute to use to uniquely identify the user. For example, 'uid' means we will use the LDAP 'uid' attribute to uniquely identify the user. LDAP Username Attribute: uid

	
Etape	Description
6	Cliquer sur “Save and Test LDAP Server Settings” pour vérifier la connexion au serveur LDAP.
	Représentation
	

4.7.3.3. Test de connexion avec un utilisateur LDAP

Une fois la configuration terminée, nous pouvons tester la connexion à *Jellyfin* avec un compte utilisateur créé dans l'annuaire LDAP:

Etape	Description
1	Se déconnecter de <i>Jellyfin</i> , puis tenter de se connecter avec les identifiants LDAP: Utilisateur “tcretard” et le mot de passe défini dans <i>phpLDAPadmin</i> .

	Représentation
	
Etape	Description
2	La connexion réussit et l'utilisateur LDAP accède à la médiathèque <i>Jellyfin</i> avec le film transféré précédemment.
	Représentation 

5. Difficultés rencontrées

Au cours de ce projet, quelques difficultés ont été rencontrées, notamment lors de la configuration de la communication entre les deux stacks *Jellyfin* et LDAP. En effet, une fois l'utilisateur créé dans LDAP, il m'était impossible de me connecter à Jellyfin. Après plusieurs

recherches, j'ai trouvé qu'il fallait simplement enlever le "LDAP Search Filter" lors de la configuration du Plugin:

```
LDAP Search Filter:  
(memberOf=CN=JellyfinUsers,DC=contoso,DC=com)  
LDAP search filter to limit user searches.
```

Il m'a fallu également me rendre compte de la nécessité de la création du "cn=users_group", pour pouvoir créer les utilisateurs.

6. Conclusion

Ce projet *Docker* réalisé dans le cadre du BTS SIO m'a permis de mettre en pratique de nombreuses compétences liées à la conteneurisation. J'ai d'abord installé et configuré *Docker* sur une machine virtuelle Debian, avant de manipuler des images et des conteneurs de manière progressive.

Puis, j'ai découvert *Portainer*, qui m'a permis de gérer visuellement les conteneurs sans passer uniquement par la ligne de commande, ce qui est bien plus pratique. Grâce à *Docker Compose*, j'ai pu déployer des stacks complets comme *Nextcloud* ou encore un serveur *Nginx* couplé à *MariaDB* et *Adminer*.

La partie la plus complexe, et aussi la plus intéressante, a été de faire communiquer deux stacks distincts: *Jellyfin* pour la gestion des médias, et *OpenLDAP* pour la gestion centralisée des utilisateurs. En créant un réseau *Docker* commun entre les deux stacks et en configurant le plugin LDAP dans *Jellyfin*, j'ai pu permettre à un utilisateur créé dans *phpLDAPadmin* de se connecter directement sur *Jellyfin* et d'accéder à la médiathèque.

Ce projet m'a aussi appris à faire face aux difficultés, notamment sur la configuration du réseau entre conteneurs et les paramètres du plugin LDAP, ce qui m'a forcé à comprendre en profondeur comment ces technologies fonctionnent ensemble.